

VOLUME 06 / RUST BACKEND MASTERY

Web 基礎と Topcoat 0.11 完全ガイド

HTTP の要求と応答から、Topcoat のページ・ルーター・view!・フォーム・アセット・セッションまでを、動かして確かめる

第6巻: Web 基礎と Topcoat 0.11 完全ガイド

この巻で身に付けること

第5巻までで、Rust の文法・所有権・設計・async を学びました。この巻では、それを使って **Web のサーバー**を作ります。ブラウザから届いた「この画面を見せて」という頼みに、Rust のプログラムが HTML を作って返す仕組みです。

Web のサーバーは、いきなり道具（フレームワーク）から始めると、何が起きているのかが見えにくくなります。次のような場面で、困ることがよくあります。

- ブラウザでは何も表示されないのに、どこが悪いのか見当がつかない
- 「404」「405」「303」のような数字が返ってきたが、それが何を意味するのか分からない
- フォームを送ったあとに再読み込みしたら、同じデータがもう1件できてしまった
- 日本語の文字が、途中で切れて表示される

そこで、この巻の最初の単元では、道具を使わずに、**標準ライブラリだけ**で小さなサーバーを書き、HTTP の要求と応答が「ただの文字のやり取り」であることを、自分の目で確かめます。そのうえで、UNIT 02 から、RustNote でも使っている Web フレームワーク **Topcoat 0.11** の使い方を、1つずつ学びます。

到達目標

この巻を終えると、次のことができるようになります。

- HTTP の要求（メソッド・パス・ヘッダー・本文）と応答（状態コード・ヘッダー・本文）の形を説明し、`curl.exe -i` や `-v` の表示を読める
- 状態コード（200・303・403・404・405 など）と、GET・POST の使い分けを説明できる
- Topcoat 0.11 で、ページ（`#[page]`）・ルーター・レイアウト・URL の部品の読み取り・`view!` とコンポーネントを使った画面を作れる
- フォームを受け取り、成功したら 303 で GET の画面へ移す（PRG）形を作れる
- 共有の状態（`app_context`）・アセット（CSS の束）・エラー・クッキー・セッション・ほかのサイトからの送信の守りを、Topcoat の仕組みで扱える

学習地図

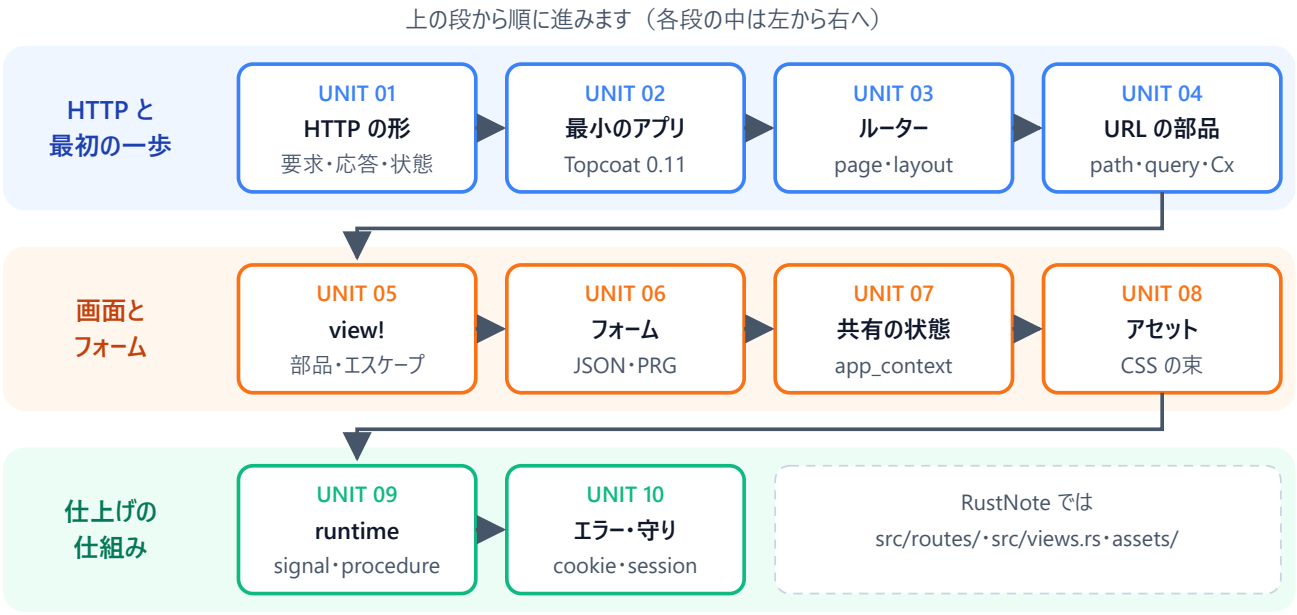


図6-1: 第6巻の学習地図。道具を使わずに HTTP の形を確かめてから、Topcoat で画面とフォームを作り、共有の状態・アセット・エラーと守りの仕組みで仕上げる

単元	テーマ	ひとことで言うと
UNIT 01	HTTP の要求・応答・状態コード	標準ライブラリだけのサーバーで、HTTP がただの文字だと確かめる
UNIT 02	Topcoat 0.11 の最小のアプリ	ページを1つ作って起動する
UNIT 03	ルーター・page・route・layout	URL ごとに処理を分け、共通の外枠で包む
UNIT 04	URL の部品（path・query）と Cx（準備中）	`/notes/{id}` や `?page=2` を型にして読む
UNIT 05	view!・コンポーネント・HTML のエスケープ（準備中）	画面を部品に分け、入力を安全に表示する
UNIT 06	フォーム・JSON・PRG（準備中）	送られてきた値を型にし、303 で移る
UNIT 07	app_context・memoize（準備中）	DB などを全部の処理で共有し、同じ問い合わせを1回にする
UNIT 08	アセット・CSS・束（準備中）	`asset!` と `topcoat asset bundle`
UNIT 09	runtime・signal・procedure・shard（準備中）	ブラウザの中で動く部分と、その守り方
UNIT 10	エラー・クッキー・セッション・CSRF（準備中）	400・404・リダイレクトと、ログインの仕組みの土台

> **お知らせ:** この版には UNIT 01〜03 を収録しています。UNIT 04〜10 は準備中で、次の更新から順に追加します。以前の版（Topcoat 0.5 の版）の PDF は、この版がそろうまで残してあります。Topcoat は 0.5 から 0.11 までに書き方が大きく変わっているので、以前の版のコードは、0.11 では動かないことがあります。

前提知識（先に読んでおく巻）

- **第2巻（Rust 基礎）**：`String` と `&str`、`match`、`Vec`、`for`。特に、日本語の文字列の「文字数」と「バイト数」の違い（第2巻 UNIT 05）。UNIT 01 のエラー1で、この違いが原因の事故を見ます。
- **第4巻（設計）**：`Result` と `?`、標準ライブラリの入出力（`std::io`）。
- **第5巻（async）**：UNIT 02 から使う `#[tokio::main]` と `async fn`。

この巻の進め方

各単元では、次の順番で進めてください。

1. **予想する:** コマンドを実行する前に、どの状態コードが返り、何が表示されるかを予想します。
2. **そのまま動かす:** サーバーを起動し、もう1つの PowerShell の窓から `curl.exe` で要求を送ります。
3. **両方の窓を見る:** サーバーの窓（届いた要求）と、curl の窓（返ってきた応答）を、並べて見比べます。
4. **1か所だけ変える:** 「よくあるエラーと読み方」の節のとおりに書き換えて、何が変わるかを見ます。

> **ヒント:** この巻では、サーバーを動かす窓と、`curl.exe` を打つ窓の2つの PowerShell を使います。サーバーを止めるときは、サーバーの窓で Ctrl+C を押します。止め忘れたまま、別のサーバーを同じポートで起動しようとすると、エラーになります。

使うツールと版

この巻のコードは、すべて次の環境で実際にコンパイル・実行し、実行結果とエラーメッセージは実際の出力をそのまま載せています（パソコンのユーザー名などを含む部分は `...` に置き換えています）。

項目	版・内容
OS	Windows 11（ARM64）。コマンドは PowerShell を主に、WSL / macOS / Linux は補足で示します
Rust	**1.99 以上**（本書シリーズの基準。Topcoat 0.11 が 1.99 以上を求めます）。動作確認は rustc 1.99.1 / cargo 1.99.1（`aarch64-pc-windows-msvc`）
edition	2024
curl	8.21.0（Windows に入っている `curl.exe`。PowerShell では `curl` だけだと別のコマンドになるので、必ず `curl.exe` と打ちます）
topcoat	0.11.0（2026-10-24 に出た版。UNIT 02 から使います。UNIT 01 では使いません）

練習用のプロジェクトを用意する

UNIT 01 では、外のクレートを1つも使いません。練習用のプロジェクトを1つ作り、`src/bin/` に、単元ごとの小さなサーバーを足していきます。

```
cargo new http_lab
cd http_lab
# このフォルダでは Rust 1.99.1 を使う（第13巻 UNIT 01）
Set-Content rust-toolchain.toml "[toolchain]`nchannel = `"1.99.1`""
New-Item -ItemType Directory src\bin
```

`src/bin/u01_raw.rs` のように置いたファイルは、`cargo run --bin u01_raw` で、それぞれ別のプログラムとして動かせます。

UNIT 01: HTTP の要求・応答・状態コード

この単元のゴール

- HTTP の**要求**（要求行・ヘッダー・空行・本文）と**応答**（状態行・ヘッダー・空行・本文）の形を、実際に届いた文字で説明できる
- 標準ライブラリだけの小さなサーバーを動かし、`curl.exe -i` と `-v` の表示を読める
- 状態コード（200・301・303・403・404・405）が、それぞれ何を伝えるのかを説明できる
- GET（見るだけ）と POST（変える）の違いと、POST の後に 303 で GET の画面へ移す（PRG）理由を説明できる

たとえ話でつかむ

役所の窓口に、申請書を出す場面を考えます。

- **申請書**のいちばん上には、「何をしたいか」（閲覧したい・登録したい）と「どの書類について」（住民票・印鑑登録）を書きます。その下に、名前や連絡先のような**添え書き**を並べ、1行空けてから、**中身**（登録する内容）を書きます。
- 窓口の人は、申請書を読んで、**返事の紙**を返します。いちばん上には「受け付けました」「その書類はありません」「その窓口ではできません」のような**結果の番号**が書いてあり、その下に添え書き、1行空けて中身（書類の写しなど）が続きます。
- 「登録」の申請が終わったら、窓口の人は「控えは、隣の閲覧の窓口で受け取ってください」と案内します。登録の窓口に何度も並び直すと、同じ登録が二重にされてしまうからです。

Web のブラウザとサーバーのやり取り（HTTP）も、同じ形をしています。

役所の窓口	HTTP
申請書のいちばん上（何を・どの書類について）	**要求行**（メソッドとパス。`GET /hello HTTP/1.1`）
申請書の添え書き	**ヘッダー**（`Host:`・`User-Agent:`・`Content-Length:` など）
1行空けてから書く中身	**本文**（フォームの値など。GET には、ふつう無い）
返事の紙の結果の番号	**状態コード**（`200 OK`・`404 Not Found` など）
閲覧の窓口・登録の窓口	**GET**（見るだけ）・**POST**（変える）
「控えは閲覧の窓口で」	**303 See Other**（POST の後に、GET の画面へ移す。PRG）

> **用語:** HTTP（エイチティーティーピー。HyperText Transfer Protocol：ブラウザとサーバーが、要求と応答をやり取りするときの決まり）

> **用語:** 要求（ようきゅう。request：ブラウザなどからサーバーへの頼み。リクエストとも言う）／応答（おうとう。response：サーバーからの返事。レスポンスとも言う）

> **用語:** 状態コード（じょうたいコード。status code：応答の1行目にある3桁の数。結果の種類を表す。ステータスコードとも言う）

図で理解する

要求と応答は、どちらも「1行目・ヘッダー・空行・本文」の4つの部分でできています。

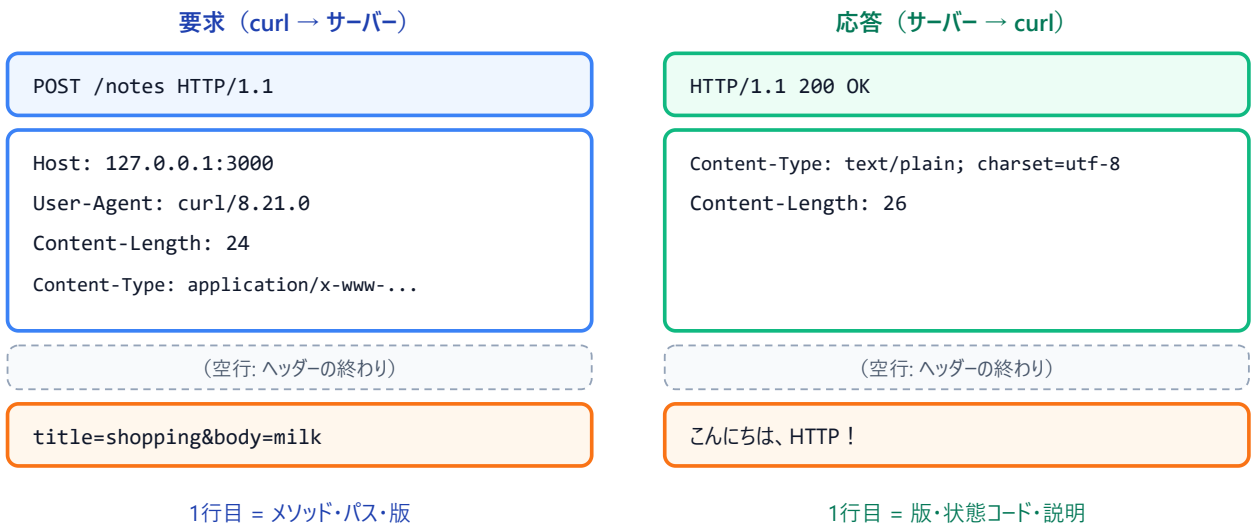


図6-2: HTTP の要求と応答の形。どちらも「1行目・ヘッダー・空行・本文」でできている。この単元の例で、実際に届いた・返した文字

- **1行目**だけが、要求と応答で違います。要求は「メソッド（`GET`・`POST` など）・パス（`/notes`）・版」、応答は「版・状態コード（`200`）・その説明（`OK`）」です。
- **ヘッダー**は「名前: 値」の行の並びです。**空行**（何も書いていない1行）で、ヘッダーが終わったことを知らせます。
- **本文**は、空行の後ろです。どこまでが本文かは、ヘッダーの `Content-Length`（本文の**バイト数**）で伝えます（エラー1）。

状態コードは、最初の数字で、おおまかな種類が分かります。



図6-3: 状態コードの種類。最初の数字で、うまくいった（2）・別の場所へ（3）・頼み方が違う（4）・サーバーの失敗（5）が分かる

まず動かす最小コード

例1-1: 届いた要求を、そのまま表示するサーバー

> **▼ このコードがやること（先に日本語で）：** `127.0.0.1:3000` で接続を待ちます。接続が来たら、要求の1行目とヘッダーを空行まで1行ずつ読み、`Content-Length` があれば、その長さだけ本文も読みます。読んだ要求をそのまま画面に表示し、「こんにちは、HTTP！」という文字を、HTTP の応答の形にして返します。外のクレートは使いません。

```rust
//! HTTP の要求と応答が「ただの文字」であることを確かめる、とても小さなサーバー。
//! 届いた要求をそのまま表示し、決まった文字を返す（標準ライブラリだけで書く）。
//! 実行: cargo run --bin u01_raw

use std::io::{BufRead, BufReader, Read, Write};
use std::net::{TcpListener, TcpStream};

/// 1つの接続を処理する: 要求を読んで表示し、応答を返す
fn handle(mut stream: TcpStream) -> std::io::Result<()> {
    let mut reader = BufReader::new(&stream);

    // 1行目（要求行）とヘッダーを、空行が来るまで1行ずつ読む
    let mut head = Vec::new();
    loop {
        let mut line = String::new();
        reader.read_line(&mut line)?;
        let line = line.trim_end().to_string();
        if line.is_empty() {
            break;
        }
        head.push(line);
    }

    // Content-Length のヘッダーがあれば、その長さ（バイト数）だけ本文を読む
    let mut length = 0;
    for line in &head {
        if let Some((name, value)) = line.split_once(':')
            && name.eq_ignore_ascii_case("content-length")
        {
            length = value.trim().parse().unwrap_or(0);
        }
    }
    let mut body = vec![0; length];
    reader.read_exact(&mut body)?;

    println!("--- 届いた要求 ---");
    for line in &head {
        println!("{line}");
    }
    if !body.is_empty() {
        println!("（空行）");
        println!("{}", String::from_utf8_lossy(&body));
    }

    // 応答を「文字列」として組み立てて送り返す。Content-Length は本文のバイト数
    let reply = "こんにちは、HTTP！\n";
    let response = format!(
        "HTTP/1.1 200 OK\r\nContent-Type: text/plain; charset=utf-8\r\nContent-Length:
{}\r\n\r\n{}",
```

```
        reply.len(),
        reply
    );
    stream.write_all(response.as_bytes())?;
    Ok(())
}

fn main() -> std::io::Result<()> {
    let listener = TcpListener::bind("127.0.0.1:3000")?;
    println!("http://127.0.0.1:3000 で待っています（Ctrl+C で終了）");
    // 接続が来るたびに、1つずつ処理する
    for stream in listener.incoming() {
        handle(stream?)?;
    }
    Ok(())
}
```

練習用のプロジェクトの `src/bin/u01_raw.rs` として保存し、1つ目の PowerShell の窓で起動します。

```
cargo run --bin u01_raw
```

2つ目の PowerShell の窓で、`curl.exe` を使って、GET の要求と POST の要求を1回ずつ送ります。`-i` は、応答の1行目とヘッダーも表示する指定、`-d` は、本文にフォームの値を入れて POST で送る指定です。

```
curl.exe -i "http://127.0.0.1:3000/hello?name=ada"
curl.exe -i -d "title=shopping&body=milk" http://127.0.0.1:3000/notes
```

▼ 実行結果（curl の窓）

```
HTTP/1.1 200 OK
Content-Type: text/plain; charset=utf-8
Content-Length: 26

こんにちは、HTTP！
```

```
HTTP/1.1 200 OK
Content-Type: text/plain; charset=utf-8
Content-Length: 26

こんにちは、HTTP！
```

▼ 実行結果（サーバーの窓。3つ目の要求は、問3の `curl.exe -v` で送ったものです）

```
http://127.0.0.1:3000 で待っています（Ctrl+C で終了）
--- 届いた要求 ---
GET /hello?name=ada HTTP/1.1
Host: 127.0.0.1:3000
User-Agent: curl/8.21.0
Accept: */*
--- 届いた要求 ---
POST /notes HTTP/1.1
Host: 127.0.0.1:3000
User-Agent: curl/8.21.0
Accept: */*
Content-Length: 24
Content-Type: application/x-www-form-urlencoded
（空行）
title=shopping&body=milk
--- 届いた要求 ---
GET /hello?name=ada HTTP/1.1
Host: 127.0.0.1:3000
User-Agent: curl/8.21.0
Accept: */*
```

> **注意:** PowerShell 5.1 で、curl の窓の日本語が `縺薙ｓ縺ｫ縺。縺ｯ` のように化けるときは、最初の `curl.exe` の前に `[Console]::OutputEncoding = [System.Text.Encoding]::UTF8` を実行します。サーバーが返した UTF-8 の文字を、PowerShell が別の文字コードで読んでいるのが原因です。

▼ コードを1つずつ分解して解説

解説1: 要求は、ただの文字の並び

- サーバーの窓には、curl が送ってきた文字が、そのまま表示されました。1行目の `GET /hello?name=ada HTTP/1.1` が要求行で、その下の `Host:`・`User-Agent:`・`Accept:` がヘッダーです。curl は、何も指定しなくても、この3つのヘッダーを付けて送ります。
- URL の `?name=ada` の部分（クエリ）は、パスの一部として要求行に入っています。
- POST のほうには、`Content-Length: 24` と `Content-Type: application/x-www-form-urlencoded`（フォームの形の本文です）が加わり、空行の後ろに、本文の `title=shopping&body=milk` が届きました。24 は、この本文のバイト数です。

解説2: 応答も、自分で書いた文字

- curl の窓の `HTTP/1.1 200 OK` から `こんにちは、HTTP！` までは、`format!` で作った文字列そのものです。行の終わりは `\r\n`（改行の前に「行の先頭へ戻る」を付けた2文字）で、HTTP の決まりです。
- このサーバーは、どんなパスやメソッドが来ても、同じ応答を返します。`/notes` に POST しても、何も保存していません。「どのパスに、どう答えるか」を決めるのが、次の例1-2と、UNIT 02 から使うフレームワーク（Topcoat）の役目です。

解説3: TcpListener は、接続を1つずつ受け取る

- `TcpListener::bind("127.0.0.1:3000")` は、「この PC の 3000 番の窓口で、接続を待つ」という意味です。`127.0.0.1` は「この PC 自身」を表す特別なアドレスで、ほかの PC からはつながりません。
- `listener.incoming()` は、接続が来るたびに1つずつ取り出します。このサーバーは、1つの接続を処理し終えるまで、次の接続を受け付けません。たくさんの人が同時に使うサーバーでは、async（第5巻）で同時にさばきます。Topcoat は、その部分を引き受けてくれます。

もう一歩: 実務に近い形

例1-2: メソッドとパスで、返す状態コードを変える

> **▼ このコードがやること（先に日本語で）**: 要求行を空白で区切って、メソッドとパスを取り出します。その組み合わせで、200（トップページ）・405（使えないメソッド）・403（見られないページ）・301（引っ越したページ）・303（伝言を受け取ったので、一覧を見てください）・404（無いページ）を返し分けます。`POST /messages` で受け取った伝言は、プログラムの中の `Vec` に貯め、`GET /messages` で一覧にします。

```rust
//! 要求のメソッドとパスを見て、いろいろな状態コードを返すサーバー（標準ライブラリだけで書く）。
//! 実行: cargo run --bin u01_status

use std::io::{BufRead, BufReader, Read, Write};
use std::net::{TcpListener, TcpStream};

/// 届いた要求（要求行・本文）
struct Request {
    method: String,
    path: String,
    body: String,
}

/// 要求を読む: 1行目（要求行）・ヘッダー・本文
fn read_request(stream: &TcpStream) -> std::io::Result<Request> {
    let mut reader = BufReader::new(stream);
    let mut first = String::new();
    reader.read_line(&mut first)?;
    // 「GET /messages HTTP/1.1」を空白で区切る
    let mut parts = first.split_whitespace();
    let method = parts.next().unwrap_or("").to_string();
    let path = parts.next().unwrap_or("").to_string();

    let mut length = 0;
    loop {
        let mut line = String::new();
        reader.read_line(&mut line)?;
        let line = line.trim_end();
        if line.is_empty() {
            break;
        }
        if let Some((name, value)) = line.split_once(':')
            && name.eq_ignore_ascii_case("content-length")
        {
            length = value.trim().parse().unwrap_or(0);
        }
    }
    let mut body = vec![0; length];
    reader.read_exact(&mut body)?;
    let body = String::from_utf8_lossy(&body).to_string();
    Ok(Request { method, path, body })
}

/// 応答を組み立てて送る。HEAD のときは、ヘッダーだけを送る
fn respond(
    stream: &mut TcpStream,
    request: &Request,
    status: &str,
    headers: &[(&str, &str)],
```

```
    body: &str,
) -> std::io::Result<()> {
    let mut text = format!(
        "HTTP/1.1 {status}\r\nContent-Type: text/plain; charset=utf-8\r\nContent-
Length: {}\r\n",
        body.len()
    );
    for (name, value) in headers {
        text.push_str(&format!("{name}: {value}\r\n"));
    }
    text.push_str("\r\n");
    if request.method != "HEAD" {
        text.push_str(body);
    }
    stream.write_all(text.as_bytes())
}

fn main() -> std::io::Result<()> {
    let listener = TcpListener::bind("127.0.0.1:3000")?;
    println!("http://127.0.0.1:3000 で待っています（Ctrl+C で終了）");
    // POST /messages で受け取った伝言を、ここに貯める
    let mut messages: Vec<String> = Vec::new();

    for stream in listener.incoming() {
        let mut stream = stream?;
        let request = read_request(&stream)?;
        println!("{} {}", request.method, request.path);

        match (request.method.as_str(), request.path.as_str()) {
            // 見るだけの操作（GET と HEAD）
            ("GET" | "HEAD", "/") => {
                respond(&mut stream, &request, "200 OK", &[], "トップページです\n")?
            }
            // / は GET と HEAD でしか使えない
            (_, "/") => respond(
                &mut stream,
                &request,
                "405 Method Not Allowed",
                &[("Allow", "GET, HEAD")],
                "その方法は使えません\n",
            )?,
            ("GET", "/admin") => respond(
                &mut stream,
                &request,
                "403 Forbidden",
                &[],
                "ここは見られません\n",
            )?,
            // 引っ越したページ: 新しい場所を Location で教える
            ("GET", "/old-page") => respond(
```

```
                &mut stream,
                &request,
                "301 Moved Permanently",
                &[("Location", "/")],
                "",
            )?,
            // 伝言を受け取ったら、303 で「GET /messages を見てください」と返す
            ("POST", "/messages") => {
                let text = request.body.split_once('=').map(|(_, value)| value);
                messages.push(text.unwrap_or("").to_string());
                respond(
                    &mut stream,
                    &request,
                    "303 See Other",
                    &[("Location", "/messages")],
                    "",
                )?
            }
            ("GET", "/messages") => {
                let list = format!("伝言は {} 件: {:?}\n", messages.len(), messages);
                respond(&mut stream, &request, "200 OK", &[], &list)?
            }
            _ => respond(
                &mut stream,
                &request,
                "404 Not Found",
                &[],
                "ページが見つかりません\n",
            )?,
        }
    }
    Ok(())
}
```

`src/bin/u01_status.rs` として保存し、例1-1のサーバーを Ctrl+C で止めてから起動します。2つ目の窓から、いろいろな要求を送ります。`-X DELETE` はメソッドを DELETE にする指定、`-I` は HEAD（ヘッダーだけをください）で送る指定、`-L` は 3xx の応答が来たら `Location` の場所へ自動でたどる指定です。

```
cargo run --bin u01_status
```

▼ 実行結果（curl の窓。`###` の行は、送ったコマンドです）

```
### curl.exe -i http://127.0.0.1:3000/
HTTP/1.1 200 OK
Content-Type: text/plain; charset=utf-8
Content-Length: 25

トップページです

### curl.exe -i http://127.0.0.1:3000/nothing
HTTP/1.1 404 Not Found
Content-Type: text/plain; charset=utf-8
Content-Length: 34

ページが見つかりません

### curl.exe -i http://127.0.0.1:3000/admin
HTTP/1.1 403 Forbidden
Content-Type: text/plain; charset=utf-8
Content-Length: 28

ここは見られません

### curl.exe -i http://127.0.0.1:3000/old-page
HTTP/1.1 301 Moved Permanently
Content-Type: text/plain; charset=utf-8
Content-Length: 0
Location: /


### curl.exe -i -X DELETE http://127.0.0.1:3000/
HTTP/1.1 405 Method Not Allowed
Content-Type: text/plain; charset=utf-8
Content-Length: 31
Allow: GET, HEAD

その方法は使えません

### curl.exe -I http://127.0.0.1:3000/
HTTP/1.1 200 OK
Content-Type: text/plain; charset=utf-8
Content-Length: 25

### curl.exe -i -d message=hello http://127.0.0.1:3000/messages
HTTP/1.1 303 See Other
Content-Type: text/plain; charset=utf-8
Content-Length: 0
Location: /messages


### curl.exe -i -L -d message=world http://127.0.0.1:3000/messages
```

```
HTTP/1.1 303 See Other
Content-Type: text/plain; charset=utf-8
Content-Length: 0
Location: /messages

HTTP/1.1 200 OK
Content-Type: text/plain; charset=utf-8
Content-Length: 36

伝言は 2 件: ["hello", "world"]
```

▼ 実行結果（サーバーの窓）

```
http://127.0.0.1:3000 で待っています（Ctrl+C で終了）
GET /
GET /nothing
GET /admin
GET /old-page
DELETE /
HEAD /
POST /messages
POST /messages
GET /messages
```

▼ コードを1つずつ分解して解説

解説1: 404・403・405 は、それぞれ違う「断り方」

- `/nothing` は、そもそも無いページなので **404 Not Found** です。`/admin` は、ページはあるが見せられないので **403 Forbidden** です。
- `DELETE /` は、ページはあるが、その**メソッド**では使えないので **405 Method Not Allowed** です。405 のときは、`Allow: GET, HEAD` のヘッダーで「使えるメソッドはこれです」と教えるのが HTTP の決まりです。
- どれも「頼んだ側を直せば通る」4xx の仲間ですが、直し方が違います（404 は URL を、403 は権限を、405 はメソッドを直す）。だから、使い分けると、使う人に正しい直し方が伝わります。

解説2: 301 と 303 は、Location で行き先を教える

- `/old-page` は **301 Moved Permanently** と `Location: /` を返しました。「このページは、ずっと `/` に引っ越しました」という意味です。ブラウザや検索エンジンは、次からは新しい場所を使います。
- `POST /messages` は **303 See Other** と `Location: /messages` を返しました。「受け取りました。結果は `GET /messages` で見てください」という意味です。`-L` を付けた最後のコマンドでは、curl が `Location` をたどって、自動で `GET /messages` を送り、`伝言は 2 件: ["hello", "world"]` を受け取りました。サーバーの窓にも、`POST /messages` の後に `GET /messages` が届いています。

解説3: POST の後に 303 で GET へ移す（PRG）

- もし `POST /messages` が、303 ではなく 200 で一覧を返していたら、ブラウザの画面は「POST の結果」のままです。利用者が再読み込み（F5）を押すと、ブラウザは**同じ POST をもう一度送り**、伝言が二重に保存されます。
- 303 で GET の画面へ移しておけば、再読み込みで送り直されるのは `GET /messages`（見るだけ）なので、何も増えません。この「POST → Redirect（303）→ GET」の形を、頭文字をとって **PRG** と呼びます。RustNote のフォームも、すべてこの形です（第9巻 UNIT 02）。



図6-4: PRG（POST → Redirect → GET）の流れ。POST の返事を 303 にして GET の画面へ移すので、再読み込みで送り直されるのは見るだけの GET になる

- GET は「見るだけ」で、何回送っても DB を変えない（**安全**な）メソッドとして扱います。ブラウザは、リンクの先を前もって読み込んだり、再読み込みで送り直したりするので、GET で何かを変えると、利用者が知らないうちに変わってしまいます。

解説4: HEAD は、ヘッダーだけを返す

- `curl.exe -I`（HEAD）には、`200 OK` と `Content-Length: 25` だけが返り、本文はありませんでした。`respond` の中の `if request.method != "HEAD"` で、本文を付けないようにしているからです。HEAD は「本文の大きさや種類だけを、先に知りたい」ときに使います。

よくあるエラーと読み方

エラー1（エラーが出ない間違い）: Content-Length に文字数を入れた

例1-1の `reply.len()`（バイト数）を、`reply.chars().count()`（文字数）にしてしまった場合です（`src/bin/u01_len_bug.rs`）。

```rust
//! エラー1の例: Content-Length に、バイト数ではなく文字数を入れてしまったサーバー。
//! 実行: cargo run --bin u01_len_bug

use std::io::{BufRead, BufReader, Write};
use std::net::TcpListener;

fn main() -> std::io::Result<()> {
    let listener = TcpListener::bind("127.0.0.1:3000")?;
    println!("http://127.0.0.1:3000 で待っています（Ctrl+C で終了）");
    for stream in listener.incoming() {
        let mut stream = stream?;
        // 要求の1行目とヘッダーを、空行まで読み飛ばす
        let mut reader = BufReader::new(&stream);
        loop {
            let mut line = String::new();
            reader.read_line(&mut line)?;
            if line.trim_end().is_empty() {
                break;
            }
        }

        let reply = "こんにちは、HTTP！\n";
        // 間違い: chars().count() は「文字数」。Content-Length に入れるのは「バイト数」
        let response = format!(
            "HTTP/1.1 200 OK\r\nContent-Type: text/plain; charset=utf-8\r\nContent-
Length: {}\r\n\r\n{}",
            reply.chars().count(),
            reply
        );
        stream.write_all(response.as_bytes())?;
    }
    Ok(())
}
```

```
cargo run --bin u01_len_bug
```

別の窓から:

```
curl.exe -i http://127.0.0.1:3000/
```

```
HTTP/1.1 200 OK
Content-Type: text/plain; charset=utf-8
Content-Length: 12

こんにち
```

- **何が起きたか:** 本文が「こんにち」で切れました。エラーは、どこにも出ていません。`Content-Length: 12` は、`こんにちは、HTTP！\n` の**文字数**（12 文字）です。ところが、日本語の1文字は UTF-8 で3バイトなので、本当の本文は 26 バイトあります。curl は、`Content-Length` を信じて 12 バイトだけを本文として読み、残りを捨てました。12 バイトは、日本語4文字ぶん（こ・ん・に・ち）です。
- **英語だけで試すと気づかない:** 英数字は1文字1バイトなので、文字数とバイト数が同じになり、この間違いは見えません。日本語のアプリでは、日本語で試すことが大切です（第2巻 UNIT 05、第9巻 UNIT 03）。
- **直し方:** `Content-Length` には `reply.len()`（`String` の `len()` はバイト数）を入れます。UNIT 02 から使う Topcoat は、この長さを自動で正しく計算してくれます。

エラー2: 誰も待っていないポートに送った（curl: (7)）

サーバーを止めたまま、`curl.exe` で要求を送った場合です。

```
curl.exe -i http://127.0.0.1:3000/
```

```
curl: (7) Failed to connect to 127.0.0.1:3000 after 2029 ms: Could not connect to
server
```

- **何が起きたか:** 「127.0.0.1 の 3000 番に、約2秒試しても接続できませんでした」というエラーです。`(7)` は curl のエラーの番号で、「接続できなかった」を表します。HTTP の状態コード（404 など）は出ていません。状態コードは、サーバーが応答を返したときにだけ届くものだからです。
- **確かめること:** サーバーの窓で、`http://127.0.0.1:3000 で待っています` が表示されているか、ポートの番号が同じかを確かめます。サーバーが起動の途中でエラーになって止まっていることもあります。
- **404 との違い:** 404 は「サーバーは動いていて、そのページが無い」、curl の `(7)` は「サーバーにたどり着けない」です。どちらの段階で失敗しているかを、最初に見分けます。

エラー3: 同じポートで2つ起動した（os error 10048）

例1-1のサーバーを止めずに、別の窓から例1-2のサーバーを起動すると、`TcpListener::bind` が失敗します。Windows では `os error 10048`（そのアドレスとポートは、もう使われています）、Linux では `os error 98` のエラーになります。先に動いているサーバーを Ctrl+C で止めてから、起動し直します（RustNote で同じエラーを見た例は、第12巻 UNIT 01 のエラー2にあります）。

RustNote ではここで使う

RustNote は、例1-2のような「メソッドとパスで返事を分ける」処理を、Topcoat に任せています。RustNote の `src/routes.rs` の先頭のコメントには、ファイルと URL の対応の表があります（抜き出しています）。

```
//! | ファイル | URL |
//! |---|---|
//! | routes.rs                | GET /（トップ）、全ページ共通のレイアウト、404 の受け皿 |
//! | routes/health.rs         | GET /health |
//! | routes/signup.rs         | GET /signup・POST /signup |
//! | routes/login.rs          | GET /login・POST /login |
//! | routes/logout.rs         | POST /logout |
//! | routes/notes.rs          | GET /notes（?q= 検索・?page= ページ送り）・POST /notes |
```

- 例1-2の `match (method, path)` の1行ずつが、RustNote では1つの関数（`#[page]` や `#[page(POST)]` を付けたもの）になります。無いページの 404、使えないメソッドの 405 は、Topcoat が自動で返します。
- RustNote のフォームの POST は、成功すると必ず 303 で GET の画面へ移します（PRG）。たとえば、ノートの作成（`POST /notes`）は、303 で、作ったノートの詳細（`GET /notes/{id}`）へ移ります。

RustNote が返す状態コードの決まりは、`src/error.rs` の先頭のコメントにまとめてあります（抜き出しています）。

```
//! | AppError | ステータス | 起きる場面 |
//! |---|---|---|
//! | `InvalidTitle` など入力の検証エラー | 400 | 空のタイトル、形式の違うメール |
//! | `Unauthorized` | 401 | ログインしていない（通常はログイン画面へのリダイレクトで扱う） |
//! | `NotFound` | 404 | 存在しないノート、**他人のノート**（あること自体を隠す） |
//! | `EmailTaken` / `Conflict` | 409 | 登録済みのメール、ほかの画面で先に更新された |
//! | `Internal` | 500 | DB の故障など。画面には詳細を出さず、ログに原因を書く |
```

- 例1-2では出てこなかった **400**（入力の形が違う）・**401**（ログインが必要）・**409**（ほかの更新とぶつかった）・**500**（サーバーの中の失敗）も使っています。他人のノートを 403 ではなく 404 にしている理由は、第8巻 UNIT 01 で扱いました。

練習問題

問1: 例1-2のサーバーに、`GET /health` を足してください。`200 OK` で、本文は `{"status":"ok"}`、`Content-Type` は `application/json`（JSON の形の本文です）にします。足したら、`GET /health` と `POST /health` を送って、それぞれの応答を確かめてください。`POST /health` の応答は、HTTP の決まりとして正しいでしょうか。

問2: 「伝言を `GET /messages?add=hello` で足せる」ように作ったとします。この作りには、どんな困ったことが起きるでしょうか。解説3の内容をもとに、2つ挙げてください。

問3: 例1-1で取った `curl.exe -v` の表示（下）の中から、curl が送った要求の部分と、サーバーが返した応答の部分を見分けてください。それぞれ、どの記号で始まっていますか。また、`*` で始まる行は何を表しているでしょうか。

```
*   Trying 127.0.0.1:3000...
* Established connection to 127.0.0.1 (127.0.0.1 port 3000) from 127.0.0.1 port 55388
* using HTTP/1.x
> GET /hello?name=ada HTTP/1.1
> Host: 127.0.0.1:3000
> User-Agent: curl/8.21.0
> Accept: */*
>
* Request completely sent off
< HTTP/1.1 200 OK
< Content-Type: text/plain; charset=utf-8
< Content-Length: 26
<
{ [26 bytes data]
* Connection #0 to host 127.0.0.1:3000 left intact
```

解答と解説

問1の解答:

`match` の、`// 見るだけの操作（GET と HEAD）` の行の前に、次の腕を足します（足したものを `src/bin/u01_q1.rs` として保存しました）。`respond` は `Content-Type` を `text/plain` に決めているので、この腕では、応答の文字を自分で組み立てます。

```
            // 問1: 動いているかを外から確かめる URL。JSON で返すので Content-Type も変える
            ("GET", "/health") => {
                let body = r#"{"status":"ok"}"#;
                let text = format!(
                    "HTTP/1.1 200 OK\r\nContent-Type: application/json\r\nContent-
Length: {}\r\n\r\n{body}",
                    body.len()
                );
                stream.write_all(text.as_bytes())?
            }
```

```
cargo run --bin u01_q1
```

```
### curl.exe -i http://127.0.0.1:3000/health
HTTP/1.1 200 OK
Content-Type: application/json
Content-Length: 15

{"status":"ok"}

### curl.exe -i -X POST http://127.0.0.1:3000/health
HTTP/1.1 404 Not Found
Content-Type: text/plain; charset=utf-8
Content-Length: 34

ページが見つかりません
```

- `GET /health` には、`Content-Type: application/json` と `{"status":"ok"}` が返りました。`Content-Length: 15` は、`{"status":"ok"}` のバイト数です。`r#"..."#` は、中に `"` を含む文字列を、`\"` と書かずにそのまま書ける Rust の書き方（生の文字列リテラル）です。
- `POST /health` には **404** が返りました。`match` のどの腕にも当たらず、最後の `_` に落ちたからです。HTTP の決まりとしては、`/health` というページはあるので、**405 Method Not Allowed** と `Allow: GET` を返すのが正確です。例1-2の `/` と同じように、`(_, "/health")` の腕を足せば直せます。こうした決まりを、ページごとに自分で書き続けるのは大変です。Topcoat は、ページに付けたメソッドから、405 と `Allow` を自動で返してくれます（第9巻 UNIT 02 で、RustNote の 405 を確かめました）。
- RustNote の `GET /health` も、同じ `{"status":"ok"}` を返します。公開したあと、外から「動いているか」を確かめるのに使います（第12巻 UNIT 01）。

問2の解答:

1. **再読み込みで、伝言が増える:** `GET /messages?add=hello` の画面で再読み込みを押すと、ブラウザは同じ GET をもう一度送るので、伝言がもう1件増えます。GET は「何回送っても変わらない」ことが前提なので、ブラウザは確かめずに送り直します。
2. **リンクを踏んだだけで、伝言が増える:** GET は、リンク（`<a href="...">`）や画像（`<img src="...">`）でも送られます。ほかのサイトに `<img src="http://127.0.0.1:3000/messages?add=spam">` と書かれていたら、そのページを開いただけで、利用者のブラウザが伝言を足してしまいます。ブラウザが「先読み」（リンクの先を前もって読み込むこと）で送ることもあります。

- だから、何かを**変える**操作は POST（や PUT・DELETE）にして、GET は**見るだけ**にします。POST にしても、ほかのサイトからフォームで送らせる攻撃（CSRF）は残るので、その守りを足します（第8巻 UNIT 01、この巻の UNIT 10（準備中））。

問3の解答:

- `>` **で始まる行**が、curl が送った要求です（`> GET /hello?name=ada HTTP/1.1` から、空の `>` の行まで）。`<` **で始まる行**が、サーバーが返した応答の1行目とヘッダーです（`< HTTP/1.1 200 OK` から、空の `<` の行まで）。矢印の向き（`>` は外へ、`<` は内へ）と覚えます。
- `*` **で始まる行**は、curl が自分の動きを説明しているメモです。`Trying 127.0.0.1:3000...`（つなごうとしています）、`Established connection`（つながりました）、`Request completely sent off`（要求を送り終えました）、`Connection #0 to host 127.0.0.1:3000 left intact`（接続は閉じずに残しました）のように、HTTP の文字ではなく、その外側で起きたことを表します。
- `{ [26 bytes data]` は、本文を 26 バイト受け取ったという印です。`-v` と一緒に `-o NUL`（本文は捨てる）を付けたので、本文そのものは表示されていません。

まとめ

- HTTP の要求と応答は、どちらも「1行目・ヘッダー・空行・本文」の文字の並び。要求の1行目はメソッドとパス、応答の1行目は状態コード。本文の長さは `Content-Length` に**バイト数**で書く。
- 状態コードは、2xx（うまくいった）・3xx（別の場所へ。`Location` で行き先を教える）・4xx（頼み方が違う。404・403・405 は直し方が違う）・5xx（サーバーの失敗）。curl の `(7)` は、そもそもサーバーにたどり着けていない。
- GET は見るだけ、POST は変える。POST が成功したら 303 で GET の画面へ移す（PRG）と、再読み込みで二重に保存されない。

UNIT 02: Topcoat 0.11 の最小のアプリ

この単元のゴール

- `cargo add` で Topcoat 0.11.0 と Tokio を足し、ページが1つだけの Web アプリを作って起動できる
- `#[page("/")]`・`Router::builder().discover().build()`・`topcoat::start` の3つが、それぞれ何をしているかを説明できる
- UNIT 01 で自分で書いた処理（`Content-Length`・404・405・HEAD）を、Topcoat が代わりにしてくれることを、`curl.exe` の表示で確かめられる
- `HOST` と `PORT` の環境変数で待ち受ける場所を変え、`.await` の書き忘れ・`use` の書き忘れ・ポートの指定の間違いのエラーを読める

たとえ話でつかむ

UNIT 01 では、お店の窓口を1人で切り盛りしました。お客さんの申込書を1行ずつ読み、「その窓口ではできません」「その書類はありません」を自分で書き、返事の紙の枚数（`Content-Length`）まで自分で数えました。

UNIT 02 では、**受付の係を雇います**。

- 受付の係は、申込書を読み、どの担当に回すかを決め、担当がいなければ「その書類はありません」（404）、担当はいても方法が違えば「その方法は使えません」（405）と、決まった形で返事をしてくれます。返事の紙の枚数も、係が数えます。

- あなたがすることは、**担当者の名札**を書くことだけです。「`/` に見に来た人には、この人が答えます」と書いた名札を、担当者に付けておきます。
- 開店のとき、受付の係は、名札を付けた担当者を**全員集めて**、名簿を作ります。
- お店の**住所と窓口の番号**（`HOST` と `PORT`）は、開店の前に掲示板（環境変数）に書いておけば、係がそれを読みます。

お店	Topcoat のアプリ
受付の係	Topcoat（ルーターと、待ち受けの仕組み）
担当者に付ける名札	`#[page("/")]`（この関数は `GET /` に答える、という印）
名札の付いた担当者を集めて名簿を作る	`Router::builder().discover().build()`
開店して、窓口を開ける	`topcoat::start(router).await`
掲示板に書いた住所と窓口の番号	環境変数 `HOST` と `PORT`（無ければ `127.0.0.1` と `3000`）
決まった形の断り方	無い URL は 404、使えないメソッドは 405 と `allow`

> **用語:** Web フレームワーク（ウェブフレームワーク：要求を読み、URL ごとの処理に振り分け、応答を返す、といった決まった仕事を引き受けてくれる道具。Topcoat もその1つ）

> **用語:** ルーター（router：要求の URL とメソッドを見て、どの関数に処理させるかを決める仕組み）

図で理解する

最小のアプリの部品と、要求が届いてから応答を返すまでの流れです。

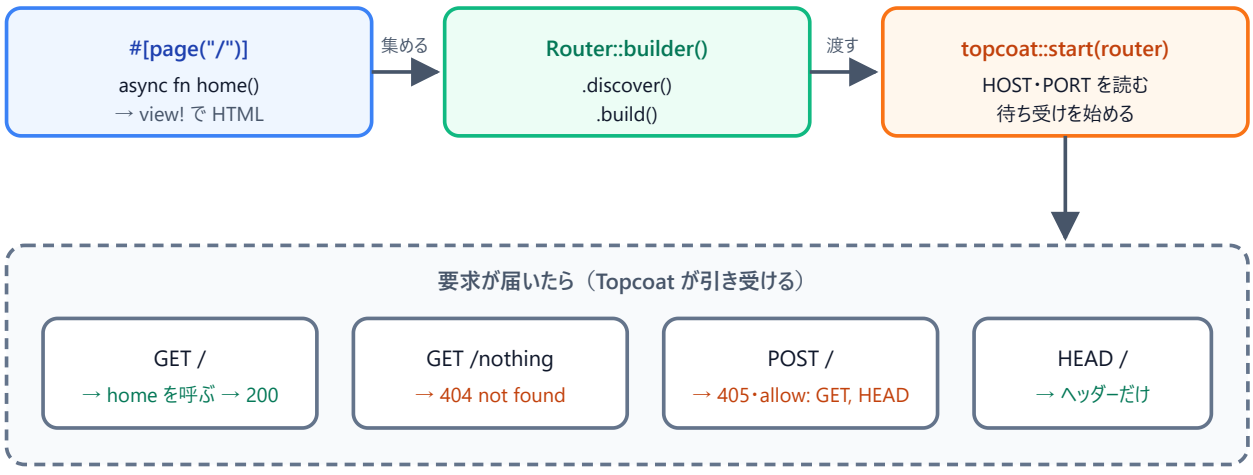


図6-5: 最小のアプリの部品。名札（#[page]）を付けた関数を discover で集めてルーターを作り、start に渡す。要求の振り分けと、404・405・HEAD の返事は Topcoat がする

- 自分で書くのは、`#[page("/")]` を付けた関数 `home` と、`main` の2行だけです。
- `discover()` は、プログラムの中の `#[page]` を付けた関数を、**全部自動で集めます**。ページを増やすときは、`#[page("/about")]` を付けた関数を足すだけで、`main` を書き換える必要はありません（問1）。
- 無い URL の 404、使えないメソッドの 405 と `allow`、HEAD のときに本文を付けないこと、`Content-Length` の計算は、どれも Topcoat がします。UNIT 01 で自分で書いた処理です。

まず動かす最小コード

例2-1: プロジェクトを作り、Topcoat と Tokio を足す

> **▼ このコマンドがやること（先に日本語で）**：新しいプロジェクトを作り、Rust の版を 1.99.1 に固定してから、`cargo add` で Topcoat（版を 0.11.0 に固定）と Tokio（非同期のランタイム）を足します。

```
cargo new web_lab
cd web_lab
Set-Content rust-toolchain.toml "[toolchain]`nchannel = `"1.99.1`""
cargo add topcoat@=0.11.0
cargo add tokio@=1.54.1 --features rt-multi-thread,macros
```

▼ 実行結果（`cargo add topcoat@=0.11.0` の表示）

```
    Updating crates.io index
      Adding topcoat =0.11.0 to dependencies
             Features:
             + asset
             + compression
             + cookie
             + discover
             + font
             + icon
             + router
             + runtime
             + serve
             + session
             + view
             22 deactivated features
    Updating crates.io index
     Locking 220 packages to highest Rust 1.99.1 compatible versions
      Adding generic-array v0.14.7 (available: v0.14.9)
```

- `+` の付いた features（`asset`・`router`・`view` など）が、既定で有効になる機能です。そのほかの機能（`tailwind`・`websocket`・`l10n` など）は既定では無効で、数が多いので、`22 deactivated features`（無効の機能が 22 個）とだけ表示されます。一覧は `cargo info -v topcoat@0.11.0` で見られます。必要なときに、自分で足します。
- `Locking 220 packages to highest Rust 1.99.1 compatible versions` は、「Rust 1.99.1 で使える範囲で、いちばん新しい版を選んで、220 個のクレートの版を `Cargo.lock` に書きました」という意味です。`Adding generic-array v0.14.7 (available: v0.14.9)` は、「もっと新しい 0.14.9 もありますが、0.14.7 にしました」というお知らせです。間接的に使う `crypto-common` が `=0.14.7` と版を固定しているからで、気にしなくて大丈夫です。
- `@=0.11.0` の `=` を付けているので、Topcoat に新しい版が出ても、勝手には上がりません（第13巻 UNIT 01）。Topcoat は、版が上がると書き方が変わることがあるので、版を上げるときは、自分で決めて、まとめて直します。この本も、0.10.0 から 0.11.0 に上げたときに、すべての巻のコードを動かし直しました。

できあがった `Cargo.toml` です（この単元の最後に、もう1つの機能を足したあとの形です。例2-3で説明します）。

```toml
[package]
name = "web_lab"
version = "0.1.0"
edition = "2024"

[dependencies]
tokio = { version = "=1.54.1", features = ["rt-multi-thread", "macros", "net"] }
topcoat = "=0.11.0"
```

例2-2: ページが1つだけのアプリ

> **▼ このコードがやること（先に日本語で）**：`GET /` に答える関数 `home` に `#[page("/")]` の名札を付け、`view!` で HTML を作って返します。`main` では、名札の付いた関数を `discover()` で集めてルーターを作り、`topcoat::start` に渡して、待ち受けを始めます。

```rust
//! Topcoat 0.11 の、いちばん小さな Web アプリ。GET / に HTML を返す。
//! 実行: cargo run

use topcoat::{
    Result,
    router::{Router, RouterBuilderDiscoverExt, page},
    view::{View, view},
};

// URL「/」に GET が来たら、この関数が HTML を作って返す
#[page("/")]
async fn home() -> Result<impl View> {
    Ok(view! {
        <!DOCTYPE html>
        <html lang="ja">
            <head>
                <meta charset="utf-8">
                <title>"はじめての Topcoat"</title>
            </head>
            <body>
                <h1>"こんにちは、Topcoat！"</h1>
                <p>"この HTML は Rust のプログラムが作りました。"</p>
            </body>
        </html>
    })
}

#[tokio::main]
async fn main() {
    // #[page] の付いた関数を、プログラム全体から集めて、ルーターを作る
    let router = Router::builder().discover().build();
    // 環境変数 HOST と PORT の場所（無ければ 127.0.0.1:3000）で待ち受ける
    topcoat::start(router)
        .await
        .expect("サーバーを起動できませんでした");
}
```

```
cargo run --bin web_lab
```

最初の1回は、Topcoat が使うクレートをコンパイルするので、少し時間がかかります（この PC では、`Compiling` の行が 164 行出て、約 22 秒でした）。

```
   Compiling web_lab v0.1.0 (...\web_lab)
    Finished `dev` profile [unoptimized + debuginfo] target(s) in 21.90s
```

サーバーの窓には、何も表示されません。 `topcoat::start` は、起動しても何も表示しないからです。止まっているように見えますが、待ち受けています。2つ目の窓から、`curl.exe` で確かめます。

```
curl.exe -i http://127.0.0.1:3000/
```

▼ 実行結果

```
HTTP/1.1 200 OK
content-type: text/html; charset=utf-8
vary: Accept
vary: accept-encoding
content-length: 228
date: Fri, 30 Oct 2026 11:31:05 GMT

<!DOCTYPE html><html lang="ja"><head><meta charset="utf-8"><title>はじめての
Topcoat</title></head><body><h1>こんにちは、Topcoat！</h1><p>この HTML は Rust のプログラム
が作りました。</p></body></html>
```

▼ コードを1つずつ分解して解説

解説1: #[page("/")] は、URL の名札

- `#[page("/")]` は、「この関数は `GET /` に答える」という印です。`#[page]` は、ふつうの関数を、ページの処理に変える**属性マクロ**です。何も指定しなければ GET に答え、HEAD にも自動で答えます。
- 関数の名前（`home`）は、URL には関係ありません。URL を決めるのは、`#[page(...)]` の中の文字列です。
- 戻り値の `Result<impl View>` は、「HTML になる何か（`View`）を返す。失敗することもある」という意味です。`view!` の中には、HTML をほぼそのまま書けます。文字は `"..."` で囲みます（UNIT 05（準備中））。

解説2: discover は、名札を自動で集める

- `Router::builder().discover().build()` は、プログラムの中にある `#[page]` の関数を全部集めて、URL の名簿（ルーター）を作ります。
- `discover` は、`RouterBuilderDiscoverExt` という trait のメソッドです。だから、`use topcoat::router::RouterBuilderDiscoverExt` が要ります（エラー2）。

解説3: Topcoat が付けてくれたヘッダー

- 応答のヘッダーは、`content-type: text/html; charset=utf-8`（HTML です。文字は UTF-8 です）・`content-length: 228`・`date`（応答を作った時刻、UTC）のほかに、`vary` が2行付いています（問3）。UNIT 01 では、`Content-Length` を自分で数えて、日本語で間違えました。Topcoat は、本文のバイト数を正しく数えてくれます。
- ヘッダーの名前が小文字（`content-type`）になっています。HTTP のヘッダーの名前は、大文字と小文字を区別しない決まりなので、`Content-Type` と同じ意味です。

- 本文の HTML は、`view!` に書いたとおりの1行になっています。字下げや改行は、`view!` が取り除いて、短くしています。

▼ Topcoat が引き受けてくれる返事

UNIT 01 の例1-2で、自分で書いた返事を、Topcoat が返すか確かめます。

```
curl.exe -i http://127.0.0.1:3000/nothing
curl.exe -i -X POST http://127.0.0.1:3000/
curl.exe -I http://127.0.0.1:3000/
```

▼ 実行結果

```
### curl.exe -i http://127.0.0.1:3000/nothing
HTTP/1.1 404 Not Found
content-type: text/plain; charset=utf-8
content-length: 9
date: Fri, 30 Oct 2026 11:31:05 GMT

not found

### curl.exe -i -X POST http://127.0.0.1:3000/
HTTP/1.1 405 Method Not Allowed
allow: GET, HEAD
content-length: 0
date: Fri, 30 Oct 2026 11:31:05 GMT



### curl.exe -I http://127.0.0.1:3000/
HTTP/1.1 200 OK
content-type: text/html; charset=utf-8
vary: Accept
vary: accept-encoding
content-length: 228
date: Fri, 30 Oct 2026 11:31:05 GMT
```

- 無い URL（`/nothing`）は `404 Not Found` で、本文は `not found` だけです。ページに名札を付けていない URL は、Topcoat が自動で 404 にします。自分の画面（ヘッダーやトップへのリンクの付いたもの）で 404 を出したいときは、`not_found!()` とレイアウトを使います（UNIT 03・10（準備中））。
- `POST /` は `405 Method Not Allowed` と `allow: GET, HEAD` です。`#[page]` は GET（と HEAD）だけに答えるので、Topcoat が「使えるのは GET と HEAD です」と教えてくれました。UNIT 01 の問1で「本当は 405 と Allow を返すべき」と書いたことを、Topcoat は何も書かなくてもしてくれます。
- `HEAD /`（`-I`）は、GET と同じヘッダー（`content-length: 228` も同じ）で、本文はありません。

もう一歩: 実務に近い形

例2-3: 待ち受けの場所を押さえてから、「待っています」と表示する

`topcoat::start` は、何も表示しません。「起動しました」と表示したくて、`start` の前に `println!` を書くと、困ったことが起きます。ポートを押さえる**前**に表示するので、ほかのプログラムがポートを使っていて起動に失敗したときも、「起動しました」と出てしまうのです（第12巻 UNIT 01 のエラー2で、RustNote が実際にそうなっているのを見ました）。

そこで、待ち受けの場所を**自分で先に押さえて**から表示し、押さえた場所を Topcoat に渡す書き方をします。`topcoat::start` の中身（Topcoat 0.11.0 の `serve.rs`）も、`TcpListener::bind` で押さえてから `serve` を呼ぶ形になっています。

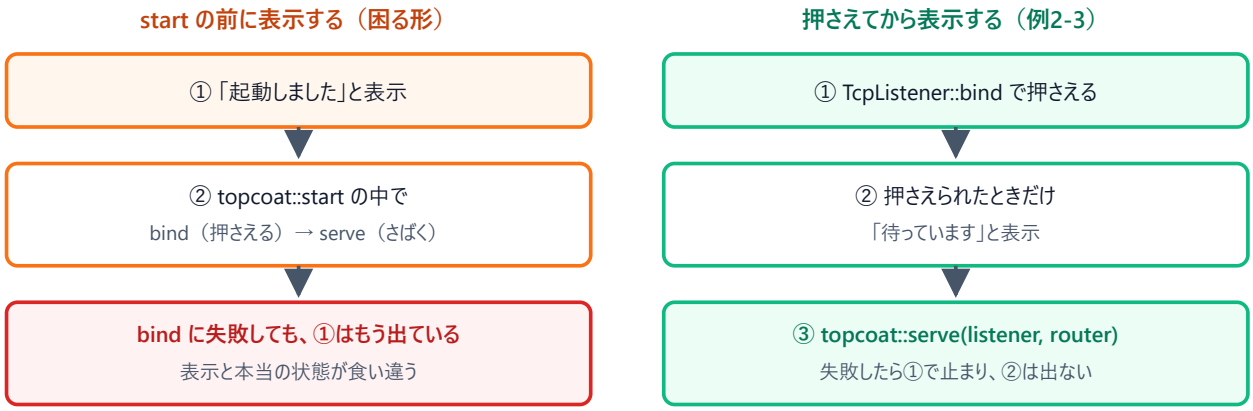


図6-6: 「起動しました」を表示する場所の違い。start は押さえるのとさばくのを中でまとめてするので、間に表示を入れられない。自分で押さえてから表示し、serve に渡せば、表示と状態が食い違わない

> ▼ **このコードがやること（先に日本語で）**: Tokio の `TcpListener::bind` で `127.0.0.1:3000` を押さえます。押さえられなかったら、そこでエラーとして終わります。押さえられたときだけ、「待っています」と表示してから、`topcoat::serve` に、押さえた場所とルーターを渡します。

```rust
//! 先に待ち受けの場所（127.0.0.1:3000）を押さえてから、「待っています」と表示する版。
//! 実行: cargo run --bin u02_serve

use tokio::net::TcpListener;
use topcoat::{
    Result,
    router::{Router, RouterBuilderDiscoverExt, page},
    view::{View, view},
};

#[page("/")]
async fn home() -> Result<impl View> {
    Ok(view! {
        <h1>"u02_serve で動いています"</h1>
    })
}

#[tokio::main]
async fn main() -> std::io::Result<()> {
    let router = Router::builder().discover().build();
    // 先にアドレスとポートを押さえる。ほかのプログラムが使っていたら、ここでエラーになって止まる
    let listener = TcpListener::bind("127.0.0.1:3000").await?;
    // 押さえられたときだけ、ここまで来る
    println!(
        "http://{} で待っています（Ctrl+C で止めます）",
        listener.local_addr()?
    );
    topcoat::serve(listener, router).await
}
```

このプログラムは `tokio::net::TcpListener` を使うので、Tokio の `net` の機能を、自分の `Cargo.toml` に足します。Topcoat がすでに `net` を有効にしているので、足さなくてもビルドは通りますが、**自分が使う機能は自分の `Cargo.toml` に書く**のが決まりです（第13巻 UNIT 01）。

```
cargo add tokio@=1.54.1 --features rt-multi-thread,macros,net
```

`src/bin/u02_serve.rs` として保存し、1つ目の窓で起動します。続けて、2つ目の窓で、同じものをもう1つ起動します。

```
cargo run --bin u02_serve
```

▼ 実行結果（1つ目）

```
http://127.0.0.1:3000 で待っています（Ctrl+C で止めます）
```

▼ 実行結果（2つ目。ポートがもう使われている）

```
Error: Os { code: 10048, kind: AddrInUse, message: "通常、各ソケット アドレスに対してプロトコル、ネットワーク アドレス、またはポートのどれか 1 つのみを使用できます。" }
```

- 1つ目は、ポートを押さえてから「待っています」と表示しました。2つ目は、`TcpListener::bind` で失敗したので、「待っています」を表示せずに、エラーで終わりました。表示と、本当の状態が食い違いません。
- `Error: Os { code: 10048, kind: AddrInUse, ... }` は、`main` が返した `Err` を、Rust が `{:?}` の形で表示したものです。`AddrInUse` は「アドレスはもう使われています」という意味です（UNIT 01 のエラー3と同じ原因です）。
- `local_addr()` は、実際に押さえた場所を返します。ポートに 0 を指定すると、空いている番号を OS が選んでくれるので、そのときに「何番になったか」を知るのにも使えます。

よくあるエラーと読み方

エラー1: start の .await を書き忘れた（E0599）

`topcoat::start(router)` の次の行の `.await` を消してしまった場合です。

```
    topcoat::start(router)
        .expect("サーバーを起動できませんでした");
```

```
cargo build
```

```
   Compiling web_lab v0.1.0 (...\web_lab)
error[E0599]: no method named `expect` found for opaque type `impl Future<Output =
Result<(), std::io::Error>>` in the current scope
  --> src\main.rs:34:10
   |
33 | /     topcoat::start(router)
34 | |         .expect("サーバーを起動できませんでした");
   | |         -^^^^^^ method not found in `impl Future<Output = Result<(),
std::io::Error>>`
   | |_________|
   |
   |
help: consider `await`ing on the `Future` and calling the method on its `Output`
   |
34 |         .await.expect("サーバーを起動できませんでした");
   |          ++++++

For more information about this error, try `rustc --explain E0599`.
error: could not compile `web_lab` (bin "web_lab") due to 1 previous error
```

- **何が起きたか**:「`impl Future<Output = Result<(), std::io::Error>>` に、`expect` というメソッドはありません」というエラーです。`topcoat::start` は async の関数なので、呼んだだけでは、まだ動いていない「これから動く仕事」（`Future`）が返るだけです。`expect` は `Result` のメソッドなので、`Future` には使えません（第5巻 UNIT 01）。
- **直し方**: `help:` のとおり、`.await` を付けます。`.await` で仕事を最後まで動かすと、`Result` が手に入り、`expect` が使えます。

エラー2: discover の trait を use し忘れた（E0599）

`use topcoat::router::{Router, RouterBuilderDiscoverExt, page}` から、`RouterBuilderDiscoverExt` を消してしまった場合です。

```
cargo build
```

```
   Compiling web_lab v0.1.0 (...\web_lab)
error[E0599]: no method named `discover` found for struct `RouterBuilder` in the
current scope
  --> src\main.rs:31:36
   |
31 |     let router = Router::builder().discover().build();
   |                                    ^^^^^^^^
   |
  ::: ...\router.rs:14:8
   |
14 |     fn discover(self) -> Self;
   |        -------- the method is available for `RouterBuilder` here
   |
   = help: items from traits can only be used if the trait is in scope
help: trait `RouterBuilderDiscoverExt` which provides `discover` is implemented but not
in scope; perhaps you want to import it
   |
 4 + use topcoat::router::RouterBuilderDiscoverExt;
   |
help: there is a method `discover_fonts` with a similar name
   |
31 |     let router = Router::builder().discover_fonts().build();
   |                                           ++++++

For more information about this error, try `rustc --explain E0599`.
error: could not compile `web_lab` (bin "web_lab") due to 1 previous error
```

- **何が起きたか:**「`RouterBuilder` に、`discover` というメソッドが見つかりません」というエラーです。`discover` は、`RouterBuilder` 自身のメソッドではなく、`RouterBuilderDiscoverExt` という trait が足しているメソッドです。trait のメソッドは、その trait を `use` しているときだけ使えます（`help: items from traits can only be used if the trait is in scope`）。
- **直し方:** 2つ目の `help:` のとおり、`use topcoat::router::RouterBuilderDiscoverExt;` を足します。3つ目の `help:` の `discover_fonts`（名前が似ているメソッド）は、別の機能なので、案内に従わないでください。

エラー3: PORT に数ではない値を入れた

PowerShell で `$env:PORT = "abc"` としてから起動した場合です。

```
$env:PORT = "abc"
cargo run --bin web_lab
```

```
thread 'main' (202724) panicked at src\main.rs:35:10:
サーバーを起動できませんでした: Custom { kind: InvalidInput, error: "PORT must be a valid
port number: invalid digit found in string" }
note: run with `RUST_BACKTRACE=1` environment variable to display a backtrace
```

- **何が起きたか:** `topcoat::start` が、環境変数 `PORT` を数として読めずに、`InvalidInput`（入力の値が不正）のエラーを返しました。`PORT must be a valid port number: invalid digit found in string` は「PORT は正しいポートの番号でなければなりません。文字列に数字ではない文字があります」です。`main` の `.expect(...)` が、このエラーを受けて panic しました。
- **直し方:** `Remove-Item Env:PORT` で環境変数を消す（既定の 3000 に戻る）か、`$env:PORT = "3001"` のように正しい番号を入れます。PowerShell で入れた環境変数は、その窓を閉じるまで残るので、ほかのプログラムにも影響することがあります。

エラー4: 実行ファイルが2つになって、cargo run が迷った

`src/bin/u02_serve.rs` を足したあとに、`--bin` を付けずに `cargo run` とした場合です。

```
cargo run
```

```
error: `cargo run` could not determine which binary to run. Use the `--bin` option to
specify a binary, or the `default-run` manifest key.
available binaries: u02_serve, web_lab
```

- **何が起きたか:**「`cargo run` は、どの実行ファイルを動かせばよいか決められません。`--bin` で指定するか、`default-run` を書いてください」というエラーです。`src/main.rs`（`web_lab`）と `src/bin/u02_serve.rs`（`u02_serve`）の2つの実行ファイルがあるからです。
- **直し方:** `cargo run --bin web_lab` のように指定します。いつも同じものを動かすなら、`Cargo.toml` の `[package]` に `default-run = "web_lab"` と書きます。RustNote の `Cargo.toml` に `default-run = "rustnote"` があるのは、このためです（第13巻 UNIT 01）。

RustNote ではここで使う

RustNote の `src/main.rs` の最後の部分です（抜き出しています）。

```
    let router = routes::build_router(AppState::new(db), assets);

    eprintln!(
        "RustNote を起動しました: http://{}",
        config::listen_address()
    );
    topcoat::start(router)
        .await
        .context("サーバーを起動できませんでした（HOST / PORT を確認してください）")?;
```

- 例2-2と同じく、`topcoat::start(router).await` で待ち受けています。`HOST` と `PORT` を読むのも同じです。公開するとき（Fly.io）は `HOST=0.0.0.0`・`PORT=8080` にします（第12巻）。
- 例2-3で見たとおり、`eprintln!` で「起動しました」を、`start` の**前**に表示しています。ポートが使われていると、この表示のあとにエラーで止まります。例2-3の形（`TcpListener::bind` で押さえてから表示し、`topcoat::serve` に渡す）にすれば直せます（RustNote のコードは変えていません）。
- RustNote のルーターは、`discover()` ではなく、`module_router!()` で作っています（`src/routes.rs` の `build_router`）。ファイルの並びをそのまま URL の並びにする書き方で、UNIT 03（準備中）で扱います。

練習問題

問1: 例2-2のアプリに、2つ目のページ `GET /about` を足してください。トップページから `/about` へのリンクも付けます。足したら、`curl.exe -i http://127.0.0.1:3000/about` で確かめてください。

問2: `$env:PORT = "3001"` としてから例2-2のアプリを起動し、`http://127.0.0.1:3001/` と `http://127.0.0.1:3000/` に `curl.exe` で要求を送ってください。それぞれ、どうなるでしょうか。

問3: Topcoat の応答には、`vary: accept-encoding` というヘッダーが付いていました。`curl.exe -i --compressed http://127.0.0.1:3000/about` で、圧縮してほしいと伝えて要求を送り、ヘッダーがどう変わるかを確かめてください。`vary` は何を伝えているのでしょうか。

解答と解説

問1の解答:

`#[page("/about")]` を付けた関数を足すだけです（足したものを `src/bin/u02_q1.rs` として保存しました）。`main` は変えません。

```rust
//! 問1の解答: 例2-1のアプリに、2つ目のページ（/about）を足したもの。
//! 実行: cargo run --bin u02_q1

use topcoat::{
    Result,
    router::{Router, RouterBuilderDiscoverExt, page},
    view::{View, view},
};

#[page("/")]
async fn home() -> Result<impl View> {
    Ok(view! {
        <!DOCTYPE html>
        <html lang="ja">
            <head>
                <meta charset="utf-8">
                <title>"はじめての Topcoat"</title>
            </head>
            <body>
                <h1>"こんにちは、Topcoat！"</h1>
                <p><a href="/about">"このアプリについて"</a></p>
            </body>
        </html>
    })
}

// 2つ目のページ。#[page] に URL を書いた関数を足すだけで、discover が見つけてくれる
#[page("/about")]
async fn about() -> Result<impl View> {
    Ok(view! {
        <!DOCTYPE html>
        <html lang="ja">
            <head>
                <meta charset="utf-8">
                <title>"このアプリについて"</title>
            </head>
            <body>
                <h1>"このアプリについて"</h1>
                <p>"第6巻 UNIT 02 の練習で作りました。"</p>
                <p><a href="/">"トップへ戻る"</a></p>
            </body>
        </html>
    })
}

#[tokio::main]
async fn main() {
    let router = Router::builder().discover().build();
    topcoat::start(router)
```

```
        .await
        .expect("サーバーを起動できませんでした");
}
```

```
cargo run --bin u02_q1
```

```
### curl.exe -i http://127.0.0.1:3000/about
HTTP/1.1 200 OK
content-type: text/html; charset=utf-8
vary: Accept
vary: accept-encoding
content-length: 258
date: Fri, 30 Oct 2026 11:31:29 GMT

<!DOCTYPE html><html lang="ja"><head><meta charset="utf-8"><title>このアプリについて
</title></head><body><h1>このアプリについて</h1><p>第6巻 UNIT 02 の練習で作りました。</p><p>
<a href="/">トップへ戻る</a></p></body></html>
```

- `discover()` が、新しい `about` も自動で集めたので、`main` を書き換えずに、`/about` が `200 OK` になりました。
- トップページのリンクは、`<a href="/about">` と URL を文字列で書いています。ページの名前を変えたときに、リンクを直し忘れても、コンパイルでは気づけません。Topcoat には、関数を指定してリンクの URL を作る `href!` があり、RustNote はそちらを使っています（第9巻 UNIT 04・この巻の UNIT 03（準備中））。

問2の解答:

```
$env:PORT = "3001"
cargo run --bin web_lab
```

別の窓から:

```
### curl.exe -s -o NUL -w "%{http_code}" http://127.0.0.1:3001/
200
### curl.exe -s -o NUL -w "%{http_code}" http://127.0.0.1:3000/
000
(curl の終了コード: 7)
```

- 3001 番は `200` でした。3000 番は、何も待ち受けていないので、`-w "%{http_code}"` の表示は `000`（応答が無い）で、curl は終了コード 7（接続できない）で終わりました（UNIT 01 のエラー2）。
- `topcoat::start` は、`PORT` を読んで、その番号で待ち受けます。プログラムを書き換えずに、待ち受ける場所を変えられるのです。確かめたら、`Remove-Item Env:PORT` で元に戻します。

問3の解答:

```
curl.exe -i --compressed http://127.0.0.1:3000/about
```

```
HTTP/1.1 200 OK
content-type: text/html; charset=utf-8
vary: Accept
vary: accept-encoding
content-encoding: gzip
transfer-encoding: chunked
date: Fri, 30 Oct 2026 11:31:29 GMT
```

- `--compressed` を付けると、curl は `Accept-Encoding`（受け取れる圧縮の形式）のヘッダーを付けて送ります。Topcoat は、本文を gzip で圧縮し、`content-encoding: gzip`（本文は gzip で圧縮してあります）を付けて返しました。圧縮したので、`content-length` の代わりに、`transfer-encoding: chunked`（本文を区切って少しずつ送ります）になっています。curl は、受け取った本文を元に戻して表示します（ここでは、ヘッダーの部分だけを載せています）。
- `vary: accept-encoding` は、「この応答は、要求の `Accept-Encoding` によって中身が変わります」という印です。途中で応答を覚えておく仕組み（キャッシュ）が、圧縮できない相手に、圧縮した応答を渡してしまわないように付けます。`vary: Accept` も同じで、要求の `Accept`（受け取れる種類）によって中身が変わることがある、という印です。
- ブラウザは、いつも `Accept-Encoding` を付けて送るので、ブラウザから見るときは、圧縮された応答が届きます。Topcoat の既定の機能（`compression`）が、これをしています。

まとめ

- `cargo add topcoat@=0.11.0` と Tokio を足し、`#[page("/")]` を付けた関数と、`Router::builder().discover().build()`・`topcoat::start(router).await` の数行で、Web アプリが動く。版は `=` で固定する（新しい版が出ても、勝手には上がらない）。
- 要求の振り分け・`Content-Length`・404・405 と `allow`・HEAD・圧縮は、Topcoat がする。`topcoat::start` は `HOST` と `PORT` を読み、起動しても何も表示しない。
- 「待っています」と表示したいときは、`TcpListener::bind` で先に押さえてから表示し、`topcoat::serve` に渡す。`.await` の書き忘れ、trait の `use` の書き忘れ、実行ファイルが2つあるときの `--bin` のエラーを読めるようにしておく。

UNIT 03: ルーター・page・route・layout

この単元のゴール

- `module_router!()` で、Rust のモジュールの並びを、そのまま URL の並びにできる（`about` → `/about`、`blog_posts` → `/blog-posts`）
- 画面を返す `#[page]` と、文字や JSON を返す `#[route(GET)]` を使い分けられる
- `#[layout]` で全ページに共通の外枠（HTML の骨組み・メニュー）を作り、入れ子のレイアウトと、処理を包む `#[layer]` の順番を説明できる
- `href!` でリンク先を作る理由と、同じ URL のページが2つあるときの起動時の panic・`(slot)` の書き忘れ・`href!` の間違いを読める

たとえ話でつかむ

大きなデパートを考えます。

- デパートは、**階**と**売り場**に分かれています。「5階の、おもちゃ売り場」のように、場所の並びで売り場が決まります。案内板は、その並びをそのまま書いています。
- どの売り場にも、**同じ建物の壁と天井**があります。売り場の人は、壁や天井を自分で作りません。建物（外枠）の中に、売り場の中身だけを並べます。
- 5階の売り場には、さらに**5階だけの飾り付け**があることもあります。建物の外枠の中に、5階の飾り付けがあり、その中に売り場があります。
- 入口には**警備員**がいて、入ってきた人と出ていった人を、全員記録します。売り場の中で何が起きても、出入りは必ず警備員の前を通ります。
- 店内の案内は、「5階のおもちゃ売り場」と**売り場の名前**で書きます。売り場が引っ越しても、名前で書いた案内なら、案内板を見れば新しい場所が分かります。

デパート	Topcoat のアプリ
階と売り場の並び	Rust の**モジュールの並び**（`mod about` → `/about`）
案内板	`module_router!()`（モジュールの並びから、URL の名簿を作る）
売り場	`#[page]`（画面を返す）と `#[route(GET)]`（文字や JSON を返す）
建物の壁と天井	`#[layout]`（どのページにも共通の、HTML の外枠）
5階だけの飾り付け	入れ子のレイアウト（`/settings` より下だけの外枠）
入口の警備員	`#[layer]`（要求の処理を、前と後ろで包む）
売り場の名前で書いた案内	`href!(about::about_page)`（関数の名前からリンク先を作る）

> **用語:** モジュールルーティング（module routing：ファイルやモジュールの並びから、URL を決める書き方。Topcoat が勧める書き方で、RustNote もこれを使う）

> **用語:** レイアウト（layout：いくつものページを包む、共通の外枠。HTML の `<html>`・`<head>`・メニューなどを1か所に書く）

> **用語:** レイヤー（layer：要求の処理の前と後ろに、決まった仕事（時間を測る・記録する など）をはさむもの）

図で理解する

例3-1のアプリの、モジュールと URL の対応です。

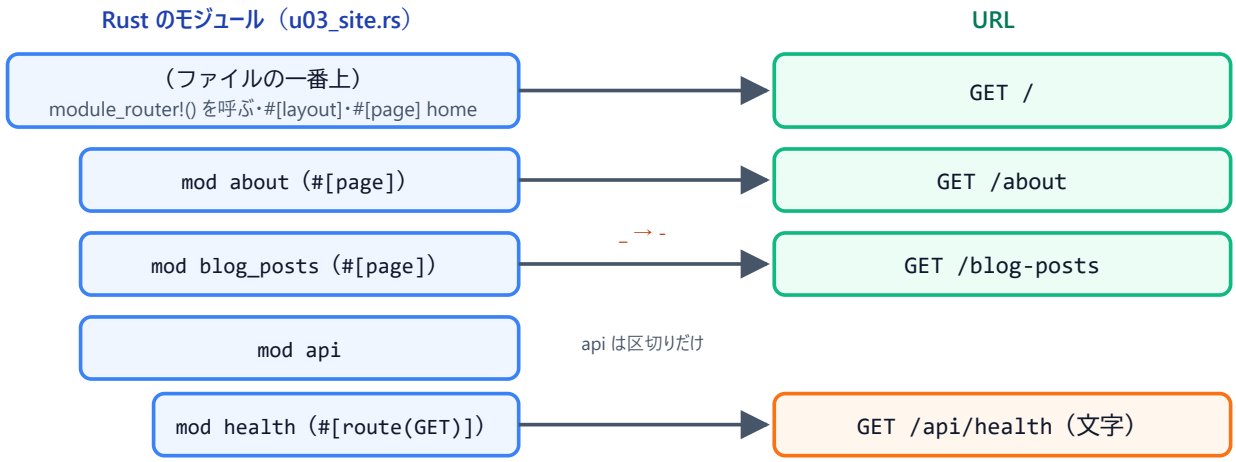


図6-7: モジュールと URL の対応。module_router!() を呼んだ場所が / になり、モジュールが1つ深くなるたびに、URL の区切りが1つ増える。モジュール名の _ は - になる

- `module_router!()` を呼んだモジュール（ここではファイルの一番上）が `/` になります。`mod about` の中の `#[page]` は `/about` に、`mod api` の中の `mod health` の `#[route(GET)]` は `/api/health` になります。
- **関数の名前は、URL に関係ありません。**`about_page` という名前でも、URL は `/about` です。
- モジュールの名前の `_` は、URL では `-` になります（`blog_posts` → `/blog-posts`）。URL では、単語を `-` でつなぐのがふつうだからです。

1つの要求が、レイヤー・レイアウト・ページを通る順番です。

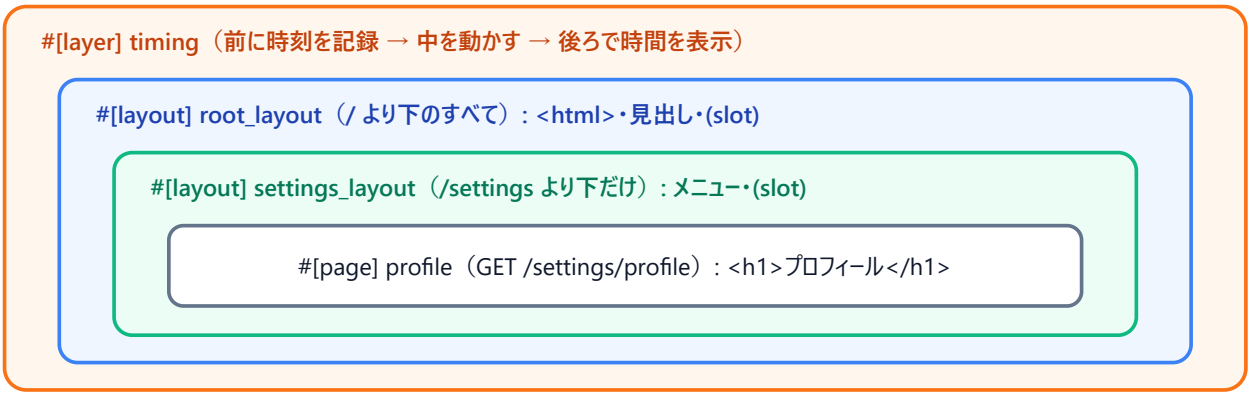


図6-8: 入れ子の順番。レイヤーがいちばん外で要求の処理を包み、その中で、広い範囲のレイアウトから狭い範囲のレイアウトへ、最後にページが入る

まず動かす最小コード

例3-1: モジュールで URL を作り、レイアウトで包む

▼ **このコードがやること（先に日本語で）**: ファイルの一番上に、全ページを包むレイアウト `root_layout` と、トップページ `home` を書きます。`mod about`・`mod blog_posts` の中にページを1つずつ、`mod api` の中の `mod health` に、文字だけを返す `#[route(GET)]` を書きます。レイアウトのメニューのリンク先は `href!` で作り、今開いているページのリンクには `aria-current="page"` と `active` のクラスを付けます。`main` では `module_router!()` でルーターを作ります。

```rust
//! 第6巻 UNIT 03: モジュールの並びを、そのまま URL の並びにする（module_router!）。
//! 実行: cargo run --bin u03_site

use topcoat::{
    Result,
    context::Cx,
    router::{Slot, href, layout, module_router, page},
    view::{View, class, view},
};

/// すべてのページを包むレイアウト（HTML の外枠と、上のメニュー）
#[layout]
async fn root_layout(cx: &Cx, slot: Slot<'_>) -> Result<impl View> {
    // 「この場所について」のリンク先と、いまそのページを開いているか
    let about = href!(about::about_page);
    let on_about = about.is_current(cx);
    Ok(view! {
        <!DOCTYPE html>
        <html lang="ja">
            <head>
                <meta charset="utf-8">
                <title>"UNIT 03 の練習"</title>
            </head>
            <body>
                <nav>
                    <a href=(href!(home))>"トップ"</a>
                    <a
                        href=(about)
                        aria-current=(on_about.then_some("page"))
                        class=(class!("nav", "active" if on_about))
                    >
                        "この場所について"
                    </a>
                    <a href=(href!(blog_posts::list))>"記事"</a>
                </nav>
                (slot)
            </body>
        </html>
    })
}

/// GET /
#[page]
async fn home() -> Result<impl View> {
    Ok(view! { <h1>"トップ"</h1> })
}

mod about {
    use super::*;
```

```
    /// GET /about
    #[page]
    pub async fn about_page() -> Result<impl View> {
        Ok(view! { <h1>"この場所について"</h1> })
    }
}

mod blog_posts {
    use super::*;

    /// GET /blog-posts（モジュール名の _ は - になる）
    #[page]
    pub async fn list() -> Result<impl View> {
        Ok(view! { <h1>"記事の一覧"</h1> })
    }
}

mod api {
    mod health {
        use topcoat::{Result, router::route};

        /// GET /api/health（HTML ではなく、文字だけを返す）
        #[route(GET)]
        async fn health() -> Result<&'static str> {
            Ok("ok")
        }
    }
}

#[tokio::main]
async fn main() {
    // このファイル（クレートの一番上）が / になる
    let router = module_router!().build();
    topcoat::start(router)
        .await
        .expect("サーバーを起動できませんでした");
}
```

UNIT 02 の `web_lab` の `src/bin/u03_site.rs` として保存し、起動します。

```
cargo run --bin u03_site
```

2つ目の窓から、4つの URL を開きます。

```
curl.exe -i http://127.0.0.1:3000/
curl.exe -i http://127.0.0.1:3000/about
curl.exe -i http://127.0.0.1:3000/blog-posts
curl.exe -i http://127.0.0.1:3000/api/health
```

▼ 実行結果

```
### curl.exe -i http://127.0.0.1:3000/
HTTP/1.1 200 OK
content-type: text/html; charset=utf-8
vary: Accept
vary: accept-encoding
content-length: 261
date: Fri, 30 Oct 2026 11:37:48 GMT

<!DOCTYPE html><html lang="ja"><head><meta charset="utf-8"><title>UNIT 03 の練習</title>
</head><body><nav><a href="/">トップ</a><a href="/about" class="nav">この場所について</a>
<a href="/blog-posts">記事</a></nav><h1>トップ</h1></body></html>

### curl.exe -i http://127.0.0.1:3000/about
HTTP/1.1 200 OK
content-type: text/html; charset=utf-8
vary: Accept
vary: accept-encoding
content-length: 303
date: Fri, 30 Oct 2026 11:37:48 GMT

<!DOCTYPE html><html lang="ja"><head><meta charset="utf-8"><title>UNIT 03 の練習</title>
</head><body><nav><a href="/">トップ</a><a href="/about" aria-current="page" class="nav
active">この場所について</a><a href="/blog-posts">記事</a></nav><h1>この場所について</h1>
</body></html>

### curl.exe -i http://127.0.0.1:3000/blog-posts
HTTP/1.1 200 OK
content-type: text/html; charset=utf-8
vary: Accept
vary: accept-encoding
content-length: 267
date: Fri, 30 Oct 2026 11:37:48 GMT

<!DOCTYPE html><html lang="ja"><head><meta charset="utf-8"><title>UNIT 03 の練習</title>
</head><body><nav><a href="/">トップ</a><a href="/about" class="nav">この場所について</a>
<a href="/blog-posts">記事</a></nav><h1>記事の一覧</h1></body></html>

### curl.exe -i http://127.0.0.1:3000/api/health
HTTP/1.1 200 OK
content-type: text/plain; charset=utf-8
content-length: 2
date: Fri, 30 Oct 2026 11:37:49 GMT

ok
```

▼ コードを1つずつ分解して解説

解説1: module_router!() は、モジュールの並びから名簿を作る

- `module_router!()` を呼んだ場所（このファイルの一番上）が `/` になり、その中の `#[page] home` が `GET /` に答えました。`mod about` の中のページは `/about`、`mod blog_posts` の中のページは `/blog-posts` です。
- `#[page]` に URL の文字列を書いていないことに注目してください。UNIT 02 では `#[page("/")]` と書きましたが、モジュールルーティングでは、**モジュールの場所が URL を決める**ので、書きません。
- `module_router!()` は、ファイルを探して回るわけではありません。Rust が `mod about` などの宣言でコンパイルしたモジュールだけが、名簿に入ります。別のファイル（`src/about.rs`）に分けたときは、`mod about;` の宣言を忘れないようにします（宣言を忘れると、そのページは 404 になります）。

解説2: #[page] は画面、#[route(GET)] は文字や JSON

- `/api/health` は、`content-type: text/plain; charset=utf-8` で、本文は `ok` の2文字だけでした。`#[route(GET)]` の関数は、`&'static str`（文字）や `Json`（第6巻 UNIT 06（準備中））などを返し、それがそのまま応答になります。**レイアウトにも包まれません。**
- `#[page]` は、`View`（画面）を返し、レイアウトに包まれます。人が見る画面は `#[page]`、プログラムやヘルスチェックが読む応答は `#[route]`、と使い分けます。
- `#[route]` は、メソッドを必ず書きます（`#[route(GET)]`・`#[route(POST)]`、いくつかなら `#[route([GET, POST])]`）。`#[page]` は、書かなければ GET です。

解説3: レイアウトは (slot) の所にページを入れる

- 4つのページのどれにも、`<nav>`（メニュー）が付いています。レイアウト `root_layout` が、`(slot)` の所に、それぞれのページの中身（`<h1>…</h1>`）を入れたからです。ページの関数には、`<html>` も `<nav>` も書いていません。
- `/about` を開いたときだけ、「この場所について」のリンクに `aria-current="page"` と `class="nav active"` が付きました。`href!(about::about_page).is_current(cx)` が「今、このリンク先を開いている」を `true` で返したからです。`aria-current="page"` は、画面の読み上げソフトに「これが今のページです」と伝える印で、`active` のクラスは、見た目（CSS）で強調するために使います。
- `class!("nav", "active" if on_about)` は、`on_about` が `true` のときだけ `active` を足して、`class` の値を作ります。`aria-current=(on_about.then_some("page"))` は、`None` のときは属性そのものを書きません（`/` の表示では、`aria-current` がありません）。

解説4: リンク先は href! で作る

- メニューのリンク先は、`"/about"` と文字列で書かずに、`href!(about::about_page)` と、**ページの関数の名前**で書きました。モジュールの名前を変えて URL が変わっても、`href!` は新しい URL を作るので、リンクを直し忘れません。名前の付け間違いは、コンパイルで分かります（エラー2）。

もう一歩: 実務に近い形

例3-2: レイアウトを入れ子にし、レイヤーで時間を測る

`/settings` より下のページにだけ、設定のメニューを付けます。全部の要求には、処理にかかった時間を表示するレイヤーを付けます。

> **▼ このコードがやること（先に日本語で）**: ファイルの一番上に、全部の要求を包むレイヤー `timing`（処理の前に時刻を覚え、処理のあとで、メソッド・パス・状態コード・かかった時間を表示する）と、外側のレイアウトを書きます。`mod settings` の中に、内側のレイアウト `settings_layout` と、`/settings` のページを、さらにその中の `mod profile` に `/settings/profile` のページを書きます。

```rust
//! 第6巻 UNIT 03 の例3-2: レイアウトを入れ子にし、レイヤーで処理の時間を測る。
//! 実行: cargo run --bin u03_nested

use topcoat::{
    Result,
    context::Cx,
    router::{
        Body, Next, Slot, layer, layout, module_router, page,
        request::{method, uri},
        response::Response,
    },
    view::{View, view},
};

/// すべての要求を包むレイヤー: 処理にかかった時間を表示する
#[layer]
async fn timing(cx: &Cx, body: Body, next: Next<'_>) -> Result<Response> {
    let started = std::time::Instant::now();
    let result = next.run(cx, body).await;
    let status = match &result {
        Ok(response) => response.status().to_string(),
        Err(error) => format!("エラー ({error}) "),
    };
    println!(
        "{} {} -> {status} ({:?}) ",
        method(cx),
        uri(cx).path(),
        started.elapsed()
    );
    result
}

/// 外側のレイアウト (/ より下の、すべてのページ)
#[layout]
async fn root_layout(slot: Slot<'_>) -> Result<impl View> {
    Ok(view! {
        <html>
            <body>
                <header>"サイトの見出し"</header>
                (slot)
            </body>
        </html>
    })
}

#[page]
async fn home() -> Result<impl View> {
    Ok(view! { <h1>"トップ"</h1> })
}
```

```
mod settings {
    use super::*;

    /// 内側のレイアウト（/settings より下のページだけ）
    #[layout]
    async fn settings_layout(slot: Slot<'_>) -> Result<impl View> {
        Ok(view! {
            <div class="settings">
                <nav>"設定のメニュー"</nav>
                (slot)
            </div>
        })
    }

    /// GET /settings
    #[page]
    async fn settings_top() -> Result<impl View> {
        Ok(view! { <h1>"設定"</h1> })
    }

    mod profile {
        use super::*;

        /// GET /settings/profile
        #[page]
        async fn profile() -> Result<impl View> {
            Ok(view! { <h1>"プロフィール"</h1> })
        }
    }
}

#[tokio::main]
async fn main() {
    let router = module_router!().build();
    topcoat::start(router)
        .await
        .expect("サーバーを起動できませんでした");
}
```

```
cargo run --bin u03_nested
```

▼ 実行結果（curl.exe -i の表示。date と vary の行は省いています）

```
### /
HTTP/1.1 200 OK
content-type: text/html; charset=utf-8
content-length: 82

<html><body><header>サイトの見出し</header><h1>トップ</h1></body></html>

### /settings
HTTP/1.1 200 OK
content-type: text/html; charset=utf-8
content-length: 139

<html><body><header>サイトの見出し</header><div class="settings"><nav>設定のメニュー</nav>
<h1>設定</h1></div></body></html>

### /settings/profile
HTTP/1.1 200 OK
content-type: text/html; charset=utf-8
content-length: 151

<html><body><header>サイトの見出し</header><div class="settings"><nav>設定のメニュー</nav>
<h1>プロフィール</h1></div></body></html>

### /settings/nothing
HTTP/1.1 404 Not Found
content-type: text/plain; charset=utf-8
content-length: 9

not found
```

▼ サーバーの窓の表示

```
GET / -> 200 OK (321.8µs)
GET /settings -> 200 OK (108.5µs)
GET /settings/profile -> 200 OK (110.3µs)
```

- `/settings/profile` の HTML は、外側のレイアウト（`<header>サイトの見出し</header>`）の中に、内側のレイアウト（`<div class="settings"><nav>設定のメニュー</nav>`）があり、その中にページ（`<h1>プロフィール</h1>`）が入っています。**範囲の広いレイアウトが外側、狭いレイアウトが内側**です。`/`（トップ）には、内側のレイアウトは付きません。
- レイヤーは、3つの要求のそれぞれで、時間を表示しました（数百マイクロ秒）。`next.run(cx, body).await` の前と後ろに書いたことが、ページの処理を**はさんで**動きます。

- `/settings/nothing`（無い URL）の 404 は、**サーバーの窓に表示されていません**。モジュールに書いたレイヤーは、そのモジュールより下で**見つかったルート**を包むので、どこにも当たらない要求は通らないのです。RustNote は、`not_found!()` を書いて、見つからない URL も自分の 404 の画面とアクセスログに通しています（UNIT 10（準備中））。

> **ヒント:** レイヤーは、ログ・時間の計測・決まったヘッダーを足す、のような「どの要求にも同じことをする」仕事に使います。「ログインしているか」の確かめは、レイヤーではなく、ページの最初で関数（RustNote の `require_user`）を呼ぶのが Topcoat の考え方です（第8巻 UNIT 01・UNIT 07（準備中））。

よくあるエラーと読み方

エラー1: 同じ URL のページが2つあって、起動した途端に止まった

`mod about` の中に、「お問い合わせ」のページを書き足してしまった場合です（`errs/e03_conflict.rs`）。

```rust
//! 同じモジュールに、GET のページを2つ書いてしまった
use topcoat::{
    Result,
    router::{module_router, page},
    view::{View, view},
};

#[page]
async fn home() -> Result<impl View> {
    Ok(view! { <h1>"トップ"</h1> })
}

mod about {
    use super::*;

    #[page]
    async fn about_page() -> Result<impl View> {
        Ok(view! { <h1>"この場所について"</h1> })
    }

    // 書き足したつもりが、同じモジュール（/about）に入れてしまった
    #[page]
    async fn contact_page() -> Result<impl View> {
        Ok(view! { <h1>"お問い合わせ"</h1> })
    }
}

#[tokio::main]
async fn main() {
    let router = module_router!().build();
    topcoat::start(router)
        .await
        .expect("サーバーを起動できませんでした");
}
```

コンパイルは通ります。起動すると、すぐに止まります。

```
thread 'main' (212584) panicked at src\bin\e03_conflict.rs:30:35:
duplicate route registered for `GET /about`
note: run with `RUST_BACKTRACE=1` environment variable to display a backtrace
```

- **何が起きたか:**「`GET /about` に、ルートが二重に登録されました」という panic です。同じモジュールの `#[page]` は、どれも同じ URL（`/about`）になるので、`about_page` と `contact_page` が、同じ `GET /about` を取り合いました。30 行目の `module_router!().build()` が、名簿を作るときに見つけて止めました。

- **直し方**:「お問い合わせ」用に `mod contact` を作り、そこにページを移します（問1）。同じモジュールに置いてよいのは、**メソッドが違う**ページだけです（RustNote の `routes/notes.rs` の `GET /notes` と `POST /notes` のように）。
- **学べること**: この間違いはコンパイルでは見つからず、**起動したとき**に分かります。名簿を作る `build()` は、起動の最初に1回だけ動くので、テスト（第11巻 UNIT 03（準備中））でルーターを組み立てるだけでも見つけられます。

エラー2: href! に、#[page] を付け忘れた関数を渡した（E0573）

`about_page` の `#[page]` を付け忘れたまま、`href!(about_page)` でリンクを作った場合です（`errs/e03_href.rs`）。

```
//! href! に、ページではない、ふつうの関数の名前を渡してしまった
use topcoat::{
    Result,
    router::{href, module_router, page},
    view::{View, view},
};

/// ページではない、ただの関数（#[page] を付け忘れた）
async fn about_page() -> Result<String> {
    Ok("この場所について".to_string())
}

#[page]
async fn home() -> Result<impl View> {
    Ok(view! {
        <h1>"トップ"</h1>
        <a href={href!(about_page)}>"この場所について"</a>
    })
}

#[tokio::main]
async fn main() {
    let router = module_router!().build();
    topcoat::start(router)
        .await
        .expect("サーバーを起動できませんでした");
}
```

```
cargo build --bin e03_href
```

```
   Compiling web_lab v0.1.0 (...\web_lab)
error[E0573]: cannot find type `about_page` in this scope
  --> src\bin\e03_href.rs:17:24
   |
17 |         <a href=(href!(about_page))>"この場所について"</a>
   |                        ^^^^^^^^^^ not found in this scope
   |
   = note: a function named `about_page` exists in another namespace

For more information about this error, try `rustc --explain E0573`.
error: could not compile `web_lab` (bin "e03_href") due to 1 previous error
```

- **何が起きたか:** 「`about_page` という**型**が見つかりません」というエラーです。`note:` は「`about_page` という名前の**関数**なら、別の名前の空間にあります」と言っています。`#[page]` は、関数と同じ名前の**型**も作り、`href!` はその型から URL を作ります。`#[page]` が無いと型が作られないので、「型が無い」と言われるのです。
- **直し方:** `about_page` に `#[page]` を付けます。`href!` に渡せるのは、`#[page]`・`#[route]` などが付いた関数だけです。
- **学べること:** 文字列で `"/about"` と書いていたら、このページが無いことにコンパイルは気づけず、リンクを押したときに初めて 404 になります。`href!` なら、**リンク先が本当にあるか**をコンパイルが確かめます。

エラー3（エラーが出ない間違い）: レイアウトに (slot) を書き忘れた

レイアウトの `view!` に、`(slot)` を書き忘れた場合です（`errs/e03_noslot.rs`。抜き出しています）。

```
#[layout]
async fn root_layout(slot: Slot<'_>) -> Result<impl View> {
    Ok(view! {
        <html>
            <body>
                <header>"サイトの見出し"</header>
            </body>
        </html>
    })
}
```

ビルドすると、エラーではなく、**警告**が1つ出ます。

```
   Compiling web_lab v0.1.0 (...\web_lab)
warning: unused variable: `slot`
 --> src\bin\e03_noslot.rs:9:22
  |
9 | async fn root_layout(slot: Slot<'_>) -> Result<impl View> {
  |                      ^^^^ help: if this is intentional, prefix it with an
underscore: `_slot`
  |
  = note: `#[warn(unused_variables)]` (part of `#[warn(unused)]`) on by default

warning: `web_lab` (bin "e03_noslot") generated 1 warning (run `cargo fix --bin
"e03_noslot" -p web_lab` to apply 1 suggestion)
    Finished `dev` profile [unoptimized + debuginfo] target(s) in 2.45s
```

起動して開くと、200 なのに、ページの中身（`<h1>トップ</h1>`）がありません。

```
### curl.exe -i http://127.0.0.1:3000/
HTTP/1.1 200 OK
content-type: text/html; charset=utf-8
content-length: 64

<html><body><header>サイトの見出し</header></body></html>
```

- **何が起きたか:** レイアウトは、受け取ったページの中身（`slot`）を、どこにも入れませんでした。どのページを開いても、見出しだけの同じ画面になります。状態コードは 200 なので、ログを見ても気づけません。
- **気づき方:** `warning: unused variable: slot`（`slot` という変数を使っていません）が、ちゃんと教えてくれていました。`help:` の「わざとなら `_slot` にしてください」に従うのではなく、**なぜ使っていないのか**を考えます。警告も、エラーと同じように読むくせを付けましょう（`cargo clippy -- -D warnings` で、警告をエラーとして扱うこともできます。第1巻 UNIT 06（準備中））。

RustNote ではここで使う

RustNote のルーターは、`src/routes.rs` の `build_router` で作っています（抜き出しています）。

```
pub fn build_router(state: AppState, assets: impl Into<AssetConfig>) -> Router {
    module_router!()
        // クッキーを読み書きできるようにする（セッションとフラッシュメッセージで使う）
        .cookies()
        // セッション（ログイン）。トークンは __Host-session クッキーで運ばれる
        .sessions(SessionConfig::default())
        // CSS を /_topcoat/assets/... で配信する
        .assets(assets)
        // どのページからも app_state(cx) で借りられるようにする
        .app_context(state)
        .build()
}
```

- 例3-1と同じく、`module_router!()` で名簿を作っています。`.cookies()`・`.sessions(…)`・`.assets(…)`・`.app_context(…)` は、ページから使う道具を、ルーターに登録しています（UNIT 07・08・10（準備中））。
- `routes.rs` の先頭のコメントには、ファイルと URL の対応の表があります。`routes/notes/id/edit.rs` が `/notes/{id}/edit` になる、のように、**ファイルの並びが、そのまま URL の並び**です。`{id}` の部分は、UNIT 04（準備中）で扱います。

同じファイルに、全ページを包むレイアウトと、アクセスログのレイヤーもあります（抜き出しています）。

```
/// アクセスログ: 「GET /notes -> 200 OK (1.2ms)」のように標準エラー出力へ書く
#[layer]
async fn access_log(cx: &Cx, body: Body, next: Next<'_>) -> Result<Response> {
    let started = std::time::Instant::now();
    let result = next.run(cx, body).await;
    // ヘルスチェックは数十秒ごとに来るので、ログに出さない
    if uri(cx).path() != "/health" {
```

- 例3-2の `timing` と同じ形です。`/health`（`routes/health.rs` の `#[route(GET)]`）だけはログに出さないように、パスで分けています。公開したあとは、Fly.io が数十秒ごとに `/health` を確かめに来るので、全部書くとログが埋まってしまうからです（第12巻 UNIT 06（準備中））。
- `root_layout` は、`<head>` に CSS へのリンクを入れ、`<body>` の中に、ヘッダー（`site_header`）と、`error_boundary`（ページで起きたエラーを、エラーの画面に変える部品。UNIT 10（準備中））で包んだ `(slot)` を置いています。

練習問題

問1: 例3-1のアプリに、「お問い合わせ」のページ（`/contact`）を足し、メニューにもリンクを足してください（エラー1の正しい直し方です）。

問2: 例3-1のアプリで、`/about/`（最後に `/` が付いている）と、`/blog_posts`（`-` ではなく `_`）を `curl.exe -i` で開くと、それぞれどうなるでしょうか。

問3: 例3-1の `/api/health` に、`curl.exe -I`（HEAD）と `curl.exe -i -X POST`（POST）で要求を送ると、どうなるでしょうか。`#[route(GET)]` には、GET しか書いていません。

解答と解説

問1の解答:

`mod contact` を作り、その中に `#[page]` の関数を書き、メニューに `href!(contact::contact_page)` のリンクを足します（`src/bin/u03_q1.rs`。足した所だけを抜き出しています）。

```
                    <a href=(href!(blog_posts::list))>"記事"</a>
                    <a href=(href!(contact::contact_page))>"お問い合わせ"</a>
```

```
mod contact {
    use super::*;

    /// GET /contact（足したページ）
    #[page]
    pub async fn contact_page() -> Result<impl View> {
        Ok(view! { <h1>"お問い合わせ"</h1> })
    }
}
```

```
cargo run --bin u03_q1
```

```
### curl.exe -i http://127.0.0.1:3000/contact
HTTP/1.1 200 OK
content-type: text/html; charset=utf-8
vary: Accept
vary: accept-encoding
content-length: 311
date: Fri, 30 Oct 2026 11:37:53 GMT

<!DOCTYPE html><html lang="ja"><head><meta charset="utf-8"><title>UNIT 03 の練習</title>
</head><body><nav><a href="/">トップ</a><a href="/about" class="nav">この場所について</a>
<a href="/blog-posts">記事</a><a href="/contact">お問い合わせ</a></nav><h1>お問い合わせ
</h1></body></html>
```

- `main` も `module_router!()` も変えずに、`/contact` が 200 になり、メニューにリンクが増えました。モジュールを1つ足すと、URL が1つ増えます。

問2の解答:

```
### curl.exe -i http://127.0.0.1:3000/about/
HTTP/1.1 308 Permanent Redirect
location: /about
content-length: 0
date: Fri, 30 Oct 2026 11:37:49 GMT



### curl.exe -i http://127.0.0.1:3000/blog_posts
HTTP/1.1 404 Not Found
content-type: text/plain; charset=utf-8
content-length: 9
date: Fri, 30 Oct 2026 11:37:49 GMT

not found
```

- `/about/` は `308 Permanent Redirect` で、`location: /about` でした。Topcoat は、最後の `/` があるかないかだけが違う URL を、**書いたとおりの形**（ここでは `/` なし）へ移します。`308` は「ずっと、こちらに移りました。同じメソッドで送り直してください」という意味です（UNIT 06 の 303 と違い、POST は POST のまま送り直されます）。ブラウザは、自動で `/about` を開き直します。
- `/blog_posts` は 404 です。URL は `/blog-posts` だけで、`_` の形は用意されません。リンクを `href!` で作っていれば、この取り違えは起きません。

問3の解答:

```
### curl.exe -I http://127.0.0.1:3000/api/health
HTTP/1.1 200 OK
content-type: text/plain; charset=utf-8
content-length: 2
date: Fri, 30 Oct 2026 11:37:49 GMT

### curl.exe -i -X POST http://127.0.0.1:3000/api/health
HTTP/1.1 405 Method Not Allowed
allow: GET, HEAD
content-length: 0
date: Fri, 30 Oct 2026 11:37:49 GMT
```

- HEAD は 200 で、GET と同じヘッダー（`content-length: 2`）で本文がありません。`GET` を書いたルートには、Topcoat が HEAD にも自動で答えます。
- POST は `405 Method Not Allowed` と `allow: GET, HEAD` でした。UNIT 02 の `#[page]` と同じく、書いていないメソッドは、Topcoat が 405 で断ってくれます。

まとめ

- `module_router!()` を呼んだモジュールが `/` になり、`mod` が1つ深くなるたびに URL の区切りが1つ増える。関数の名前は URL に関係なく、モジュール名の `_` は `-` になる。
- 画面は `#[page]`（レイアウトに包まれる）、文字や JSON は `#[route(GET)]`（そのまま応答になる）。リンクは `href!` で関数の名前から作り、無いページへのリンクをコンパイルで見つける。
- レイアウトは広い範囲が外側、狭い範囲が内側に入れ子になり、`(slot)` の所にページが入る。レイヤーは処理を前と後ろではさむ。同じ URL・同じメソッドのページが2つあると、起動したときに panic で止まる。

この巻のまとめ

この版に収録した UNIT 01〜03 で、次のことができるようになったかを確かめましょう。できたものに印を付けてください。

UNIT 01: HTTP の要求・応答・状態コード

- ☐ HTTP の要求と応答が「1行目・ヘッダー・空行・本文」の文字の並びであることを、標準ライブラリだけのサーバーで確かめられる
- ☐ 要求の1行目（メソッド・パス・版）と、応答の1行目（版・状態コード・説明）を読める
- ☐ `Content-Length` に入れるのが文字数ではなくバイト数である理由と、まちがえたときに本文が切れることを説明できる
- ☐ 200・301・303・403・404・405 の違いと、405 のときに `Allow` を付ける理由を言える
- ☐ GET は見るだけ、POST は変えるという使い分けと、POST の後に 303 で GET へ移す PRG の理由を説明できる
- ☐ `curl.exe` の `-i`・`-d`・`-X`・`-I`・`-L`・`-v` を使い分け、`-v` の `>`・`<`・`*` の行を読み分けられる
- ☐ curl の `(7)`（たどり着けない）と、404（たどり着いたが、ページが無い）を見分けられる

UNIT 02: Topcoat 0.11 の最小のアプリ

- ☐ `cargo add topcoat@=0.11.0` と Tokio で、ページが1つのアプリを作って起動できる。`=` で版を固定する理由を言える
- ☐ `#[page("/")]`・`Router::builder().discover().build()`・`topcoat::start(router).await` の役割を説明できる
- ☐ 404・405 と `allow`・HEAD・`Content-Length`・圧縮を、Topcoat が代わりにしてくれることを、`curl.exe` で確かめられる
- ☐ `HOST` と `PORT` の環境変数で待ち受ける場所を変えられる。`PORT` が数でないときのエラーを読める
- ☐ `topcoat::start` が何も表示しないことと、`TcpListener::bind` で押さえてから表示して `topcoat::serve` に渡す書き方を説明できる
- ☐ `.await` の書き忘れ（E0599）、`RouterBuilderDiscoverExt` の `use` の書き忘れ（E0599）、実行ファイルが2つあるときの `--bin` のエラーを読める

UNIT 03: ルーター・page・route・layout

- ☐ `module_router!()` を呼んだモジュールが `/` になり、`mod about` が `/about`、`mod blog_posts` が `/blog-posts` になることを説明できる
- ☐ 画面を返す `#[page]` と、文字や JSON を返す `#[route(GET)]` を使い分け、`#[route]` がレイアウトに包まれないことを言える
- ☐ `#[layout]` の `(slot)` の所にページが入ることと、入れ子のレイアウトの順番（広い範囲が外側）を説明できる
- ☐ `#[layer]` で処理の前と後ろをはさみ、見つからない URL はモジュールのレイヤーを通らないことを確かめられる
- ☐ リンク先を `href!` で作る理由と、`is_current` で今のページのリンクに `aria-current` を付ける方法を言える
- ☐ 同じ URL のページが2つあるときの起動時の panic、`#[page]` を付け忘れたときの `E0573`、`(slot)` を書き忘れたときの警告を読める

> **お知らせ:** この版には UNIT 01〜03 を収録しています。UNIT 04（URL の部品と Cx）・UNIT 05（view!・コンポーネント・HTML のエスケープ）・UNIT 06（フォーム・JSON・PRG）・UNIT 07（app_context・memoize）・UNIT 08（アセット・CSS・束）・UNIT 09（runtime・signal・procedure・shard）・UNIT 10（エラー・クッキー・セッション・CSRF）は準備中で、次の更新から順に追加します。それまでの間、Topcoat を使った完成したアプリの例は、教材のフォルダの `Rust/rustnote/src/routes.rs` から読み始められます。

用語ミニ辞典

この巻（UNIT 01〜03）に出てきた用語です。

UNIT 01 の用語

用語	意味
HTTP	ブラウザとサーバーが、要求と応答をやり取りするときの決まり
要求（request）	サーバーへの頼み。1行目（メソッド・パス・版）・ヘッダー・空行・本文でできている
応答（response）	サーバーからの返事。1行目（版・状態コード・説明）・ヘッダー・空行・本文でできている
メソッド	要求の種類。GET（見る）・POST（変える）・HEAD（ヘッダーだけ見る）・DELETE（消す）など
パス	要求の1行目の、`/notes` のような場所の部分。`?` の後ろはクエリ
ヘッダー	「名前: 値」の行の並び。空行で終わる
`Content-Length`	本文のバイト数。受け取る側は、この数だけ本文を読む
状態コード	応答の1行目の3桁の数。2xx 成功・3xx 別の場所へ・4xx 頼み方が違う・5xx サーバーの失敗
`Location`	3xx の応答で、行き先を教えるヘッダー
`Allow`	405 の応答で、使えるメソッドを教えるヘッダー
PRG	POST → Redirect（303）→ GET。再読み込みで二重に送られないようにする形
安全なメソッド	何回送っても、サーバーの中のデータを変えないメソッド（GET・HEAD）
`TcpListener`	決まったアドレスとポートで、接続を待つ標準ライブラリの型
`127.0.0.1`	「この PC 自身」を表すアドレス。ほかの PC からはつながらない
curl の `(7)`	サーバーに接続できなかったときの curl のエラー

UNIT 02 の用語

用語	意味
Web フレームワーク	要求を読み、URL ごとの処理に振り分け、応答を返す、といった決まった仕事を引き受けてくれる道具
ルーター	要求の URL とメソッドを見て、どの関数に処理させるかを決める仕組み
`#[page("/")]`	この関数は `GET /`（と HEAD）に答える、という Topcoat の属性マクロ
`discover()`	プログラムの中の `#[page]` の関数を、全部自動で集める（`RouterBuilderDiscoverExt` を use して使う）
`topcoat::start`	環境変数 `HOST`・`PORT`（無ければ 127.0.0.1:3000）で待ち受けを始める。起動しても何も表示しない
`topcoat::serve`	自分で押さえた待ち受けの場所（`TcpListener` など）とルーターを受け取って、要求をさばく
`view!`	HTML をほぼそのまま書いて、ページの中身を作るマクロ
`vary`	この応答が、要求のどのヘッダーによって変わるかを伝えるヘッダー
`content-encoding: gzip`	本文を gzip で圧縮してある、という印
`default-run`	実行ファイルが2つ以上あるとき、`cargo run` で動かすものを決める `Cargo.toml` の設定

UNIT 03 の用語

用語	意味
モジュールルーティング	モジュールの並びから URL を決める書き方。`module_router!()` を呼んだモジュールが `/` になる
`#[route(GET)]`	文字や JSON などを、そのまま応答にする関数の印。レイアウトに包まれない
`#[layout]`	いくつものページを包む共通の外枠。`(slot)` の所にページの中身が入る
`Slot`	レイアウトが受け取る、内側のページ（または内側のレイアウト）の中身
`#[layer]`	要求の処理を前と後ろではさむ関数。`next.run(cx, body).await` で中を動かす
`href!`	`#[page]` などの関数の名前から、リンク先の URL を作るマクロ。無いページはコンパイルで見つかる
`is_current`	そのリンク先を、今開いているかどうかを返すメソッド
`aria-current="page"`	画面の読み上げソフトに「これが今のページです」と伝える属性
kebab-case	単語を `-` でつなぐ書き方（`blog-posts`）。モジュール名の `_` は URL では `-` になる
308 Permanent Redirect	「ずっと、こちらに移りました。同じメソッドで送り直してください」という返事
E0573	型が来るはずの所で、型が見つからないエラー（ここでは `#[page]` を付け忘れた）